

ARISTOTLE UNIVERSITY OF THESSALONIKI

MASTER'S THESIS

Graph Embedding and Node Features for Drug-Target Interaction Prediction

Author:
Georgios
MICHOULIS

Supervisor:
Dr. Grigorios
TSOUMAKAS

A thesis submitted in fulfillment of the requirements for the degree of Master of Data and Web Science

in the

Department of Computer Science

April 4, 2022

Acknowledgements

I would first like to thank my Master's Thesis supervisor, Associate Professor Dr. Grigorios Tsoumakas, of the Department of Informatics of the Aristotle University of Thessaloniki, for his guidance and mentoring. Our frequent communication and cooperation, allowed me to overcome any obstacles in my way towards concluding this research. Additionally, I'd want to express my gratitude to Dr. Bin Liu and PhD student Dimitris Papadopoulos, without whom I would not have been able to complete my study. They provided me with valuable support whenever I needed it, were dedicated to our meetings, and were patient with me.

Furthermore, I would like to extend my gratitude towards the administration and the faculty members of the Master's program, for designing an engaging and fascinating, yet challenging curriculum.

In addition, I have to thank my family and friends for their inexhaustible and unconditional love and support throughout a tumultuous year.

Last but not least, I would like to thank and express my gratitude to Dr. Andy Brook, my mentor from DeepMind, for his valuable help, insightful ideas, patience, and support, and most importantly, for being a reliable friend throughout the MSc studies. Moreover, I want to especially thank DeepMind for this opportunity through this scholarship, which helped me fulfill my studies and be present during my darkest times. This thesis is dedicated to the memory of my father, who were the most supportive person in my life, and endorsed all the decisions I ever made, even to choose and study this MSc. He will always be in my heart.

Abstract

One of the most important tasks in the drug discovery process is to identify possible drug-target interactions (DTIs). The research and development of novel drugs frequently consumes billions of dollars and more than a decade of effort with high failure frequency. As a result, it is critical for pharmaceutical companies to identify novel drug–target interactions (DTIs) by leveraging known DTIs. Existing drugs have well-known qualities and are confirmed to be safe. However, biochemical tests for identifying novel DTIs have limitations in terms of coverage and throughput. As a result, computer approaches for the prediction of DTIs have garnered considerable interest. Computational prediction of DTI has been a prominent topic in the bioinformatics sector for the past decade, and it has substantially sped up drug development. Existing methods can be divided into two categories: network-based and classification-based methods.

This thesis focuses on the network-based category that employs methods from the graph representation learning field. We propose two different models fulfilling the tasks of the drug-target binding affinity (DTA) prediction (regression) and the drug target interaction prediction (classification), utilizing graph embeddings and graph convolutions.

Under these specifications, a data collection process is described for retrieving drug and protein features from public biological databases. Specifically, we extract SMILES of drugs and amino acid sequences of proteins in FASTA form. Then using several programming tools, we transform the drugs to graphs (where nodes correspond to atoms and edges to atom bonds), and the proteins to vectors using Word2Vec. The DTA approach leverages graph convolutions and 1D convolutions to transform drug and target features respectively, and concatenates the two outputs to consequently predict the binding affinity. In the DTI approach, drug features are extracted using graph convolutions, and then both drugs and targets are used to form a heterogeneous graph. This graph is then transformed by a graph auto-encoder, which generates the predicted interactions.

Finally, we thoroughly present the models and their results, and compare them with recent state-of-the-art methods, which demonstrates the effectiveness of our approaches.

Περίληψη

Η διαδικασία ανακάλυψης φαρμάκων βασίζεται στον εντοπισμό πιθανών αντιδράσεων φαρμάκου-στόχου (Drug Target Interaction - DTIs). Βιολογικός στόχος μπορεί να είναι οποιοδήποτε βιομόριο (πρωτεΐνη) που έχει ένα υποδοχέα, επιτρέποντάς του να συνδέεται με ένα ενδογενές πρόσδεμα (ligand) ή ένα χημικό μόριο (φάρμακο). Η έρευνα και η ανάπτυξη νέων φαρμάκων συχνά καταναλώνει δισεκατομμύρια δολάρια και περισσότερο από μια δεκαετία προσπάθειας και συχνά αποτυγχάνει. Οπότε, είναι σημαντικό για τις φαρμακοβιομηχανίες να εντοπίζουν νέες αλληλεπιδράσεις φαρμάκου-στόχου (DTIs) αξιοποιώντας γνωστά DTIs. Ωστόσο, οι πόροι των βιοχημηχανιών είναι περιορισμένοι. Ως αποτέλεσμα, οι προσεγγίσεις υπολογιστών για την πρόβλεψη των DTI έχουν συγκεντρώσει σημαντικό ενδιαφέρον. Οι υπάρχουσες μέθοδοι χρήσης υπολογιστών μπορούν να χωριστούν σε δύο κατηγορίες: μεθόδους που βασίζονται σε γράφους (graph-based) και σε μεθόδους ταξινόμησης (classification-based).

Σε αυτή την διπλωματική επικεντρωνόμαστε σε λύσεις που αξιοποιούν γράφους από το πεδίο του graph representation learning. Προτείναμε δύο διαφορετικά μοντέλα που το ένα προβλέπει την συνεχή τιμή μίας αντίδρασης (παλινδρόμηση) και το άλλο προβλέπει την αλληλεπίδραση στόχου φαρμάκου ως δυαδική τιμή (ταξινόμηση), χρησιμοποιώντας συνελίξεις γράφων (graph convolution) .

Σύμφωνα με αυτές τις προδιαγραφές, περιγράφεται μια διαδικασία συλλογής δεδομένων για την ανάκτηση χαρακτηριστικών φαρμάκων και πρωτεϊνών από δημόσιες βάσεις δεδομένων. Για τα φάρμακα συγγεντρώσαμε τα SMILES και για τις πρωτεΐνες τις αλληλουχίες αμινοξέων. Στη συνέχεια, μετατρέψαμε τα φάρμακα σε γράφους και τις πρωτεΐνες τις διανυσματοποιήσαμε με τον αλγόριθμο Word2Vec. Το μοντέλο της πρόβλεψης του DTA χρησιμοποιεί συνελίξεις γράφων για να μετασχηματίσει τους γράφους και τα διανύσματα, και η συνένωση τους χρησιμοποιείται για την πρόβλεψη συνεχούς τιμής της αντίδρασης (binding affinity). Τα χαρακτηριστικά των φαρμάκων για το μοντέλο DTI, εξάγονται χρησιμοποιώντας συνελίξεις γράφων και στη συνέχεια τα φάρμακα και οι στόχοι σχηματίζουν ένα ετερογενές γράφο. Ύστερα, ο γράφος με την χρήση του auto-encoder παράγει τις προβλεπόμενες αντιδράσεις.

Τέλος, τα μοντέλα και τα αποτελέσματά τους παρουσιάστηκαν αναλυτικά με παραστικό τρόπο. Συγκρίναμε τα αποτελέσματα με πρόσφατες καινοτόμες εργασίες, σχετικες με τις δύο προσεγγίσεις μας. Τα αποτελέσματα της σύγκρισης παρουσιάζονται και στο τελευταίο μέρος της παρούσας διπλωματικής εργασίας.

Contents

Acknowledgements iii

Abstract v

Περίληψη vii

1 Introduction 1
1.1 Contributions . . . 2
1.2 Thesis Overview . . . 3

2 Graph Representation Learning 5
2.1 Graphs . . . 5
2.1.1 Multi-relational Graphs . . . 6
2.1.2 Machine Learning . . . 6
2.2 Traditional Approaches . . . 8
2.2.1 Basic node statistics and features . . . 8
2.2.2 Graph-level measures . . . 9
2.2.3 Neighborhood Overlap Detection . . . 10
2.3 Embeddings . . . 12
2.3.1 Encoder-Decoder . . . 12
2.4 Graph Neural Networks . . . 15
2.4.1 Neural Message Passing . . . 15
2.4.2 Graph Convolution Networks . . . 18
2.4.3 Graph Pooling . . . 19
2.5 Practical Implementation . . . 20
2.5.1 Efficiency Concerns and Parameter Sharing . . . 21
2.6 Graph Generation . . . 22
2.6.1 Deep Learning Generative Methods . . . 23
2.6.1.1 Varioational Autoencoders . . . 23
2.6.1.2 Adversarial Approaches . . . 24

3 Drug Target Interaction Prediction 27
3.1 Introduction to Drug Target Interaction . . . 27
3.2 Related Work . . . 30
3.2.1 DTI with Graph Representation Learning . . . 30

4 Methodology **45**
4.1 Data Preparation 45
4.1.1 Drug and Target Collection 46
4.1.2 Drug and Protein Feature Extraction 46
4.2 Machine Learning 48
4.2.1 GENGRAM-DTA 48
4.2.2 LP-DTI 50
4.3 Results and Discussion 54
4.3.1 Metrics 54
4.3.2 GENGRAM-DTA 57
4.3.3 LP-DTI 57
4.3.4 Comparison with Related work 58

5 Conclusions and Future Work **61**
5.1 Conclusions 61
5.2 Future work 62

List of Figures

2.1 GraphSAGE Overview [41] 18

3.1 Drug Target Interaction process [57] 28

4.1 Amino Acids 47
4.2 OGB Purpose 48
4.3 GENGRAM-DTA model Architecture 49
4.4 Abstract overview of the GENGRAM-DTA 51
4.5 DeepGCN Architecture 51
4.6 Abstract overview of the EncoderDecoder 52
4.7 Abstract architecture of LP-DTI 53
4.8 Area Under the Curve example [104] 55
4.9 Train - Validation set learning curves 57
4.10 LP-DTI learning curves. 58

List of Tables

4.1 Metrics Overview 56
4.2 GENGRAM-DTA comparison table 59
4.3 LP-DTI comparison table 59

Chapter 1

Introduction

The therapeutic properties of drugs are derived from their capacity to bind to receptors on the surface of cells or to enzymes within cells. A drug most frequently attaches to (interacts with) the receptor site of a protein (target), where it can either imitate a physiological function (Agonist) or decrease the action of another agonist (Antagonist) by blocking the binding of other agonists. Rather than receptors, some drugs attack enzymes, which are responsible for regulating the pace of chemical processes. Inhibitors and inducers are two types of drugs that target enzymes. As a result, finding drug-target interactions is a critical stage in the drug development process (DTIs).

However, finding drug target interactions using wet-lab research (in-vitro) is an expensive and time-consuming approach. As a result of the foregoing, there is a need for dependable computational approaches (in silico) that anticipate such drug-target interactions, therefore limiting the search area for wet-lab validations.

Drug discovery is a very expensive process, since it involves several distinct stages, during which the new drug undergoes multiple clinical studies before reaching the market. The overall cost of producing a new drug is estimated to be approximately 1.8 billion, with a timeline of close to 10 years to reach patients. Thus, repositioning or repurposing a drug is seen as an efficient technique for treating a condition with an already found treatment. This is feasible because any given molecule may have several targets and frequently does. Recognizing this, scientists have embraced polypharmacology (one drug, numerous targets) as a therapeutic idea in recent years, replacing the more conventional dogma of "one drug, one target, one illness."

Additionally, drug side effects are linked to the drug's various targets. As a result, DTI prediction can aid in the process of drug development, repositioning, and side effect prediction, as well as potentially lowering drug costs. There are around 90 million known compounds, however only a tiny percentage have interaction profiles established [1]. Additionally, the human genome has roughly 30,000 genes, of which less than 400 are exploited as therapeutic targets [2]. Given the vast number of unknown DTIs, computational approaches for DTI

prediction appear to be an immensely helpful tool for reducing the time and expense associated with drug development.

1.1 Contributions

In this thesis, we explore the task of drug target interaction prediction, using graph embeddings and node features. Our contributions mainly focus on extracting and creating drug and target features that are used to predict binding affinity (regression) and drug target interaction (classification).

The feature-based approaches represent both the drugs and the targets as fixed-length feature vectors, and the known drug–target pairs are classified as positive or negative. The DTI prediction is then turned into a problem of binary classification. Support vector machines, random forests, and conditional random fields are all examples of machine learning techniques that may be utilized directly for prediction [3].

Recently, network embedding approaches [4] have demonstrated superior performance in the interpretation of network data [5], and have been applied for DTI prediction [6]. Embedding a network is often referred to as graph embedding. Nodes such as drugs and targets may all be turned into low-dimensional vectors that describe their properties and can be utilized directly to infer new DTIs. Matrix factorization, random walk, and deep learning are the three primary approaches for network embedding.

Furthermore, there are several approaches which are using deep learning models for predicting drug–target binding affinity [7]. However, in these cutting-edge deep learning models, drugs are represented as strings for computational convenience, which is not a natural representation of them because structural information about molecules is lost.

In this thesis, we propose two different approaches for the DTI and DTA prediction problems, respectively. In the drug-target binding affinity prediction model (GENGRAM-DTA) the drugs are represented as graphs where the edges are the bonding of atoms. Then layers of convolutions on graphs of drugs, concatenated with 1D convolution for protein features (which were extracted using Word2Vec and 3-Gram) are together regressed with the affinity of drug-target couples.

In our DTI prediction model, named LP-DTI, we use an autoencoder for a link prediction method, in which the drug features are extracted by graph convolutions and the protein features are generated in the same way as defined for GENGRAM-DTA. The encoder consists of two graph convolutions, while the decoder concatenates the drugs and targets and return the predicted interaction. Finally, having

the above models' scores, we compare them with other related works (baselines), and we present comparative results.

1.2 Thesis Overview

In Chapter 2 we describe all the popular graph representation learning methods, from which we will implement some. Following the theoretical background, Chapter 3 defines the Drug Target Interaction problem and also presents recent relevant works, some of which have inspired our approach. Chapter 4 describes our approaches. At first, we present our pipeline for collecting the drugs and targets and then presenting the methods of extracting the features from the drugs and the targets that will be used for both techniques. Finally, this chapter ends with the model definition for both techniques that we introduced, from both theoretical and technical point of views, which are the preprocessing methods and the training models. Subsequently, in Chapter 5 we describe the metrics that will measure the performance of our models, and we go through the different experiments conducted and present their results. These results will then be compared with other state-of-the-art related techniques and will discuss the comparison results. Eventually, in Chapter 6, we conclude our thesis and discuss some potential research directions in the future.

Chapter 2

Graph Representation Learning

This chapter will discuss graph representation learning, starting from basic knowledge of the graphs up to modern state-of-the-art deep learning methods. First, we will describe more basic origins, such as "what is a graph" and "what types of graphs exist." Then we will go through more advanced machine learning techniques and graph embeddings, and finally, we will describe neural graph net- works and other advanced and recent methods.

2.1 Graphs

Graphs are a pervasive data structure and a common way for describing complicated systems. In the broadest sense, a graph is nothing more than a collection of objects (i.e., nodes) connected by a set of interactions (i.e., edges). Graphs provide a mathematical framework upon which we may build to study, comprehend, and learn from complicated real-world systems. There has been a rise in the quantity and quality of graph-organized data available to academics during recent years.

Frank Harary [8] in 1969 defined a graph as: A graph G consists of a finite nonempty set V of points (nodes) together with a prescribed set X of q unordered pairs of distinct points of V. Each pair $x = u, v$ of points in X is a line (edge) of G, and x is said to join u and v. A more current definition of a simple graph by William L. Hamilton [9] is: A graph $G = (V, E)$ is defined by a set of nodes V and a set of edges E between these nodes. We denote an edge going from node $u \in V$ to node $v \in V$ as $(u, v) \in E$. Simple is because there is at most one edge between each pair of nodes, no edges between a node and itself, and where the edges are all undirected, i.e., $(u, v) \in E \longleftrightarrow (v, u) \in E$.

An adjacency matrix is a useful way to represent graphs $A \in \mathbb{R}^{|V| \times |V|}$. To represent a network using an adjacency matrix, the graph's nodes must be ordered in such a way that each node references a specific row and column in the adjacency matrix [9]. So, if there is an edge between two nodes, the matrix will represent it as 1, or it will represent it as 0: $A[u, v] = 1$, if $(u, v) \in E$, else $A[u, v] = 0$. Undirected graphs are

symmetric matrices, whereas directed graphs are not necessarily symmetric [9]. While the adjacency matrix of simple graphs is binary, the adjacency matrix of a weighted graph contains real values. For example, a weighted edge in a graph of protein-protein interactions might represent the affinity of a relationship between two proteins.

2.1.1 Multi-relational Graphs

Multi-relational graphs are those that have different types of edges. For example, in graphs representing drug-drug interactions, a type of edges may relate to various side effects that can occur when a pair of medicine are taken together. So, in that case, the edge notation includes also the relation type of the edge τ, e.g., $(u, t, v) \in E$, and thus an adjacency matrix A_t is defined per edge type, while the entire graph can be summarized by an adjacency tensor $A \in \mathbb{R}^{|V| \times |R| \times |V|}$, where R is the set of relations.

A subset of multi-relational graphs is *heterogeneous graphs*. Nodes in heterogeneous networks also have types, which allows us to split the set of nodes into distinct sets $V = V_1 \cup V_2 \cup \cdots \cup V_k \text{where} V_i \cap V_j = \oslash, \forall i \neq j$ [9]. Edges in heterogeneous graphs usually meet node type restrictions, meaning they only connect nodes of a specific kind. In a heterogeneous biomedical graph, the nodes may represent proteins, or drugs, or diseases etc. "Treatment" edges would only appear between drug and disease nodes. Similarly, polypharmacy-related edges would only occur between two drug nodes [9].

In many cases, a graph can have an *attribute or feature information* associated with it. Usually, these are node-level attributes represented by a real-valued matrix $X \in \mathbb{R}^{|V| \times m}$, with the nodes ordered by the adjacency matrix's ordering [9]. In heterogeneous graphs each different type of node (or edge) has its distinct type of attributes [9].

2.1.2 Machine Learning

Graphs can formulate several machine learning tasks. The first task we will discuss is *node classification*, where the goal is to predict the label y_u associated with all the nodes $u \in V$ given the true labels on a training set of nodes $V_{train} \subset V$ [9]. There are additional cases of node classification that need generalization across unconnected networks and/or that require a large number of labeled nodes (e.g., classifying the function of proteins in the interactomes of different species). The important difference in a graph is that the nodes are not independent and identically distributed [9]. The key factor in node classification is to leverage the edge similarity and assign labels to nodes based on that. More precisely, homophily is the tendency for nodes to share attributes with their neighbors in the graph (e.g., friendship network) [10]. There are

also other concepts such as structural equivalence [11], which assumes that nodes with comparable local neighborhood structures will have similar labels, as well as heterophily, which presupposes that nodes with dissimilar labels would be preferentially connected.

In this study, we focus on *link prediction* for the drug- target interaction problem. The standard setup for link prediction is that we are given a set of nodes V and an incomplete set of edges between these nodes $E_{train} \subset E$ [9]. This method uses this partial information to infer the missing or unobserved edges $E \setminus E_{train}$ (e.g., predicting interactions between compounds and proteins in a drug-target network). Link prediction can be challenging in large multi-edge graph datasets, such as biological knowledge networks [12]. Link prediction can also have a temporal aspect, where, given a snapshot of the set of links at time t, the goal is to predict the links at time $t + 1$ [13]. Additionally, similar to node classification, link prediction has several variations, including settings in which predictions must be performed across a single fixed graph and settings in which links must be predicted across multiple disjoint graphs [14]. Individual link prediction methods construct a model that classifies each connection separately. By defining the job as a collective link prediction task, structured prediction methods capture the correlation between potential linkages. Collective link prediction techniques develop a model that collectively identifies the genuine linkages from a set of all possible links.

Link prediction may alternatively be posed as a type of missing value estimate. The graph is represented here as an adjacency matrix with missing values. The objective is to complete the matrix by finding the values that are missing. This approach is frequently used in matrix factorization-based techniques [15]. Numerous techniques to link prediction have been suggested, including unsupervised approaches based on similarity measures derived on entity characteristics, or based on random walks and matrix factorization, and supervised approaches based on graphical models and deep learning. Link prediction techniques may be classified into two major groups according to the underlying network type: (1) methods to link prediction for homogeneous networks; (2) approaches to link prediction for heterogeneous networks. Approaches can be classified as topology-based, content-based, or hybrid techniques based on the type of information utilized to predict connections [16]. We will further discuss those techniques in the next section.

A notable machine learning method of graphs is *community detection*, which is an analog of unsupervised clustering. The community identification algorithm's objective is to infer latent community structures from the input graph $G = (V, E)$. Community identification has a variety of real-world applications, including identifying functional

modules in genetic interaction networks [17] and identifying fraudulent groups of users in financial transaction networks [18].

In *graph classification or regression scenarios*, the goal is to learn over graph data, but rather than making predictions over the individual components of a single graph (i.e., the nodes or the edges), the goal is to make independent predictions for each graph in a given dataset. Each graph is an independently distributed and identically labeled datapoint, and the objective is to learn a mapping from datapoints (i.e., graphs) to labels using a labeled set of training points. In a related task called *graph clustering*, the objective is to discover an unsupervised measure of similarity between graph pairings. Similar to unsupervised clustering, graph clustering is a natural extension of unsupervised clustering for graph data. The difficulty with these graph-level tasks, however, is defining meaningful characteristics that consider the relational structure of each datapoint [9].

2.2 Traditional Approaches

We will begin by reviewing fundamental graph statistics, kernel techniques, and their application to node and graph classification tasks. Following that, we'll present and explore several methods for determining the overlap between node neighbors, which serve as the foundation for powerful heuristics for relation prediction. Finally, we will conclude this background section by providing a quick overview of spectral clustering through graph Laplacians.

2.2.1 Basic node statistics and features

The first node feature to examine is *node degree*, which is usually denoted d_u for a node $u \in V$ and simply counts the number of edges incident to a node:

$$d_u = \sum_{v \in V} \mathbf{A}[u, v] \tag{2.1}$$

In general, a node's degree is a critical statistic to examine, and it is frequently one of the most important features in machine learning models applied to node-level tasks.

While node degree merely indicates how many neighbors a node has, this is not always sufficient to indicate a node's significance in a network [9]. Because node degree is not a powerful measure, several other more significant measures are based on *centrality*. An example of a more powerful measure is the eigenvector centrality. Whereas degree only counts the number of neighbors a node has, eigenvector centrality additionally counts the importance of a node's neighbors. To define the eigenvector centrality of a node e_u it needs a recurrence

relation in which the node's centrality is proportional to the average centrality of its neighbors [9]:

$$e_u = \frac{1}{\lambda} \sum_{v \in V} \mathbf{A}[u, v] e_v \forall u \in \mathcal{V} \tag{2.2}$$

where λ is a constant. Considering positive centrality values, the Perron-Frobenius Theorem shows that the vector of centrality values e corresponds to the greatest eigenvalue of A. To acquire eigenvector centrality values, power iteration is used and it can be calculated since A's leading eigenvector is known:

$$\mathbf{e}^{(t+1)} = \mathbf{A}\mathbf{e}^{(t)} \tag{2.3}$$

In general, at iteration $t \geq 1, e_{(t)}$ will contain the number of length t paths arriving at each node, obtaining a score proportional to the number of times a node is visited on infinite length paths. Additionally, another two popular node centrality measures are the following:

- Betweenness centrality. It measures how often a node lies on the shortest path between two other nodes.
- Closeness centrality. It measures the average shortest path length between a node and all other nodes.

Clustering coefficient measures how tightly clustered a node's neighborhood is. In other words, clustering coefficient quantifies the fraction of closed triangles in a node's immediate neighborhood.

2.2.2 Graph-level measures

The following methods come under the broad category of graph kernel methods, which are ways to create features for graphs or implicit kernel functions for use in machine learning models.

Weisfeiler-Lehman kernel method is an advanced level of a bag of nodes by using iterative neighborhood aggregation. This approach extracts node-level features that are more detailed than their local ego graph and then aggregate these more detailed aspects into a graph-level representation [9]. This method follows these steps:

1. First, assigns an initial label $l^{(0)}(v)$ to each node, which can also be the node degree ($l^{(0)}(v) = d_v \forall v \in V$).
2. Then, it assigns a new label to each node iteratively by hashing the multiset of existing labels in the node's neighborhood:

$$l^{(i)}(v) = \text{HASH}\left(\left\{\left\{l^{(i-1)}(u) \forall u \in \mathcal{N}(v)\right\}\right\}\right) \tag{2.4}$$

where the double-braces are used to denote a multi-set and the HASH function maps each unique multi-set to a unique new label.

3. After K re-labeling iterations, each node now has a label that summarizes its K-hop neighborhood. In other words, the WL kernel is computed by comparing two graphs' resulting label sets.

Graphlets and path-based methods Two more powerful methods to define features over graphs are the graphlets and path-based methods. The graphlet kernel [9] enumerates all conceivable graph substructures of a given size and counts their occurrences in the complete graph. Rather than enumerating graphlets, path-based techniques analyze the various types of routes that exist in the graph [9].

2.2.3 Neighborhood Overlap Detection

Neighborhood overlap detection is used for the link prediction machine learning task. Based on the simplest method, which is the measure that counts the number of neighbors that two nodes share, there are more advanced techniques ($\mathbf{S}[u, v] = |\mathcal{N}(u) \cap \mathcal{N}(v)|$, where $\mathbf{S}[u, v]$ is the relationship between nodes u and v and $\mathbf{S} \in \mathbb{R}^{\mathcal{V}|\times|\mathcal{V}|}$ denotes to similarity matrix) [9]. A shortcoming of this technique is that it ignores the proportional quantity of shared neighbors.

Local overlap measures

1. The first approach is the Sorensen index. It defines a matrix of node-node neighborhood overlaps as entries and normalizes the count of common neighbors by the sum of the node degrees, given by:

$$\mathbf{S}_{\text{Sorenson}}[u, v] = \frac{2|\mathcal{N}(u) \cap \mathcal{N}(v)|}{d_u + d_v}, \mathbf{S}_{\text{Sorenson}} \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|} \quad (2.5)$$

2. Salton index. It normalizes by the product of the degrees of u and v, can be used: $\mathbf{S}_{\text{Salton}}[u, v] = \frac{2|\mathcal{N}(u)\cap\mathcal{N}(v)|}{\sqrt{d_u d_v}}$.

3. Jaccard overlap or measure. It computes the relative number of common neighbors: $\mathbf{S}_{\text{Jaccard}}[u, v] = \frac{|\mathcal{N}(u)\cap\mathcal{N}(v)|}{|\mathcal{N}(u)\cup\mathcal{N}(v)|}$.

There are additional measures that go beyond counting common neighbors and assess their importance. These measures favor low-degree neighbors, assuming that a shared low-degree neighbor is more informative than a shared high-degree neighbor.

- The Resource Allocation (RA) index is used to calculate the reciprocal degrees of shared neighbors: $\mathbf{S}_{\text{RA}}[v_1, v_2] = \sum_{u \in \mathcal{N}(v_1) \cap \mathcal{N}(v_2)} \frac{1}{d_u}$
- The Adamic–Adar measure [19] is equal to the sum of the logarithms of the intersections of two nodes' neighbors. This encapsulates a two-hop similarity, which can produce more accurate results than straightforward one-hop approaches: $\mathbf{S}_{\text{AA}}[v_1, v_2] = \sum_{u \in \mathcal{N}(v_1) \cap \mathcal{N}(v_2)} \frac{1}{\log(d_u)}$

Local methods are restricted in that they evaluate just the neighborhoods of local nodes. *Global overlap* counts two nodes that can be members of the same community in the graph but they have no local overlap.

- The Katz index is the most fundamental measure of global overlap. To calculate the Katz index, the number of routes of all lengths between two nodes must be counted:

$$\mathbf{S}_{\text{Katz}}[u, v] = \sum_{i=1}^{\infty} \beta^i \mathbf{A}^i[u, v] \tag{2.6}$$

  Where $\beta \in \mathbb{R}^+$ is a user-defined parameter that specifies the relative importance of short vs long paths. Due to the Geometric series of matrices theorem, Katz can also be calculated as a full matrix of node-node similarity values [9].

- Leicht et al. [20] presented an improved measure based on the ratio of observed to anticipated paths between two nodes ($\frac{\mathbf{A}^i}{\mathbb{E}[\mathbf{A}^i]}$). In other words, the number of routes between two nodes is normalized. The expectation factor E is called the configuration model and assumes that we draw a random graph with the same set of degrees as our given graph [9].

- Random walk. The technique is defined as follows: the similarity between two nodes is proportional to the probability of reaching one of the two nodes through a random walk starting from the other one. The random walk can work with a variant from the Personalized PageRank algorithm [21], a random walk variant, which is defined as $\mathbf{q}_u = c\mathbf{P}\mathbf{q}_u + (1-c)\mathbf{e}_u$, where e_u is a one-hot indicator vector for node u, P the stochastic matrix, and $q_u[v]$ gives the stationary probability that random walk starting at node u visits node v. At each timestep, the c term specifies the chance that the random walk will restart at node u. Because random walks are constantly "teleported" back to the node u, the restart probability acquires a value unique to that node. Thus, the node-node random walk similarity measure is:

$$\mathbf{S}_{\text{RW}}[u, v] = \mathbf{q}_u[v] + \mathbf{q}_v[u] \tag{2.7}$$

Random walks also have many advantages over graph searching algorithms such as Breadth-First Search (BFS) or Depth First Search (DFS). Random walks are efficient in terms of both space and time. The space complexity to store each node's near neighbors is O(|E|). The temporal complexity of random walks over search-based sampling procedures is another important advantage. Random walks provide a straightforward way to boost an effective sampling rate by reusing samples across different source nodes by imposing network connectedness in the sample generation process.

Another tool that is useful to describe in this section and will be helpful for the next section with the description of low dimensional embeddings, is the *Graph Laplacians*. Adjacency matrices may describe graphs accurately. However, there are other useful mathematical matrix representations of graphs. These Laplacian representations are produced by various transformations of the adjacency matrix [9]. It may be used in conjunction with Kirchhoff's theorem to determine the number of spanning trees in a given graph. Cheeger's inequality may be used to estimate the sparsest cut of a graph using the second-lowest eigenvalue of its Laplacian [9]. The unnormalized Laplacian, defined as follows: **L= D - A**, where D is the degree matrix and A is the adjacency matrix of the graph. So, this matrix is i) symmetric, ii) L has $|V|$ non-negative eigenvalues, and iii) vector identity $\mathbf{x}^\top \mathbf{L}\mathbf{x}$ holds $\forall \mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$ [9]. There are two more popular Laplacian matrices. Both of these matrices have comparable features to the Laplacian, however, their algebraic properties are slightly different owing to the normalization. The first is the symmetric matrix ($\mathbf{L}_{\text{sym}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}}$) and the second is the random walk Laplacian ($\mathbf{L}_{\text{RW}} = \mathbf{D}^{-1}\mathbf{L}$) [9], [22].

2.3 Embeddings

In this section, we will describe some techniques which aim at encoding nodes as low-dimensional vectors. The basic idea behind node embedding approaches is to use dimensionality reduction techniques to distill the high-dimensional information about a node's graph neighborhood into a dense vector embedding. In other words, we wish to project nodes into a latent space in which geometric interactions correspond to the links (i.e., edges) in the original graph or network [23].

2.3.1 Encoder-Decoder

In general, an encoder model converts each graph node into a low-dimensional vector or embedding. The decoder model uses these low-dimensional node representations to reconstruct the original network structure. In its simplest form, the encoder is a function that maps

nodes $v \in V$ to vector embeddings $z_v \in \mathbb{R}^d$ (where z_v is the embedding for node $v \in V$). An encoder based on the shallow embedding approach, is a function that simply performs an embedding lookup based on the node ID [9] and described as $\text{ENC}(v) = \mathbf{Z}[v]$, where $Z \in \mathbb{R}^{|V| \times d}$ is a matrix storing all nodes' embedding vectors, and $Z[v]$ signifies the row of Z corresponding to node v.

The role of the decoder is to reconstruct certain graph statistics from the node embeddings that are generated by the encoder [9]. Pairwise decoders anticipate the connection or similarity between nodes. This reconstructs the connection between nodes u and v using two embeddings (z_u, z_v) [9]. The objective is to improve the encoder and decoder as: $\text{DEC}(\text{ENC}(u), \text{ENC}(v)) = \text{DEC}(\mathbf{z}_u, \mathbf{z}_v) \approx \mathbf{S}[u, v]$. $S[u, v]$ is a graph-based similarity metric. To predict if two nodes are neighbors, the basic reconstruction target is to minimize an empirical reconstruction loss over a set of training node pairs [9]:

$$\mathcal{L} = \sum_{(u,v) \in \mathcal{D}} \ell\left(\text{DEC}\left(\mathbf{z}_u, \mathbf{z}_v\right), \mathbf{S}[u, v]\right) \tag{2.8}$$

Where $\ell : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ is a loss function (e.g., mean-squared error or cross-entropy) measuring the discrepancy between the decoded similarity values (DEC) and the true values. Most approaches minimize the loss using stochastic gradient descent [24].

There are several representative *node embedding* methods based on matrix factorization (e.g., Laplacians), and random walks. Factorization-based approaches approximate a node-node similarity matrix S in a low-dimensional space, where S generalizes the adjacency matrix and represents a user-defined concept of node-node similarity [9]. The loss functions of these methods can be reduced using factorization algorithms (e.g., SVD). Two factorization-based methods are worth mentioning, the Laplacian eigenmaps and the inner-product. The first builds upon the spectral clustering ideas. Then, the decoder is based on the L2-distance between the node embeddings and the loss function weighs pairs of nodes according to their similarity in the graph [9]. The second's method decoder, as its name suggest, is based on inner-product ($\text{DEC}\left(\mathbf{z}_u, \mathbf{z}_v\right) = \mathbf{z}_u^\top \mathbf{z}_v$) and use a mean-squared error loss function. Thus, the dot product of two nodes' embeddings is assumed to be proportional to their similarity.

Random walk based embedding methods employ node embeddings to compare two nodes that tend to co-occur on brief random walks over the graph [9]. DeepWalk and node2vec [25] employ a shallow embedding technique with an inner-product decoder, like matrix factorization. These approaches differ in how they determine node similarity and neighborhood reconstruction. Rather than explicitly rebuilding A or some deterministic function of A, these methods optimize embeddings to capture random walk statistics [9]. The decoder is defined as

follow:

$$\begin{aligned} \text{DEC}\left(\mathbf{z}_u, \mathbf{z}_v\right) &\triangleq \frac{e^{\mathbf{z}_u^\top \mathbf{z}_v}}{\sum_{v_k \in \mathcal{V}} e^{\mathbf{z}_u^\top \mathbf{z}_k}} \\ &\approx p_{\mathcal{G},T}(v \mid u) \end{aligned} \tag{2.9}$$

where $p_{\mathcal{G},T}(v \mid u)$ is the probability of visiting v on a length-T random walk starting at u, with T usually defined to be in the range $T \in (2, \ldots, 10)$. The main technique for training random walk embeddings is to employ the decoder and minimize the cross-entropy loss, but both methods follow a different path. DeepWalk approximates the decoder using a hierarchical softmax, which uses a binary-tree structure to speed up processing [4]. However, node2vec approximates the decoder using a noisy contrastive technique, where the normalization factor is approximated using negative samples:

$$\mathcal{L} = \sum_{(u,v)\in\mathcal{D}} -\log\left(\sigma\left(\mathbf{z}_u^\top \mathbf{z}_v\right)\right) - \gamma \mathbb{E}_{v_n \sim P_n(\mathcal{V})}\left[\log\left(-\sigma\left(\mathbf{z}_u^\top \mathbf{z}_{v_n}\right)\right)\right] \tag{2.10}$$

The σ represents a logistic function, $P_n(V)$ represents a distribution over the set of nodes V, and $\gamma > 0$ represents a hyperparameter. In practice, $P_n(V)$ is frequently specified as a uniform distribution, with Monte Carlo sampling used to estimate the expectation. Node2Vec is still used a lot today, so it is important to provide more details for this method. Specifically, DeepWalk defines $p_{G,T}(v|u)$ solely using uniformly random walks [9], the node2vec approach introduces hyperparameters that allow the random walk probabilities to smoothly interpolate between walks that are more akin to breadth-first or depth-first search over the graph. Node2vec also uses a 2nd order random walk with two parameters p and q which guide the random walk with probability $\pi_{v,x} = \alpha_{pq}(t,x) \cdot w_{v,x}$, where

$$\alpha_{pq}(t,x) = \begin{cases} \frac{1}{p} & \text{if } d_{tx} = 0 \\ 1 & \text{if } d_{tx} = 1 \\ \frac{1}{q} & \text{if } d_{tx} = 2 \end{cases} \tag{2.11}$$

The parameters p and q, intuitively, determine the speed with which the walk explores and departs the neighborhood of beginning node u. The parameters allow the search to interpolate between BFS and DFS, reflecting an inclination for alternative node equivalence ideas. More specifically, p controls the likelihood of immediately revisiting a node in the walk and avoids 2-hop redundancy in sampling, and q allows the search to differentiate between "inward" and "outward" nodes.

2.4 Graph Neural Networks

The original Graph Neural Network (GNN) was introduced in 2005 by Merkwirth and Lengauer [26] and updated by Scarselli et al. in 2009 [27]. They proposed a basic GNN utilizing the neural message passing (neighborhood aggregation) framework as:

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)}\,\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\right) \tag{2.12}$$

where $\mathbf{W}_{\text{self}}^{(k)}, \mathbf{W}_{\text{neigh}}^{(k)} \in \mathbb{R}^{d^{(k)} \times d^{(k-1)}}$ are trainable parameter matrices and σ denotes an element-wise non-linearity (e.g., a tanh or ReLU). The bias term $\mathbf{b}^{(k)} \in \mathbb{R}^{d^{(k)}}$ is often omitted for notational simplicity, but including the bias term can be important to achieve strong performance.

2.4.1 Neural Message Passing

A GNN is defined by the fact that it employs a kind of neural message transmission in which vector messages are exchanged and modified between nodes using neural networks [28]. A hidden embedding $h^(k)_u$ corresponding to each node $u \in V$ is updated after each message-passing iteration in a GNN using information gathered from u's graph neighborhood $N(u)$, and is described as follow:

$$\begin{aligned}\mathbf{h}_u^{(k+1)} &= \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}\left(\left\{\mathbf{h}_v^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right)\right) \\ &= \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right)\end{aligned} \tag{2.13}$$

where UPDATE and AGGREGATE are arbitrary differentiable functions (i.e., neural networks) and $m_{N(u)}$ is the "message" that is aggregated from uR's graph neighborhood N(u) [9]. At each iteration k of the GNN, the AGGREGATE function takes as input the set of embeddings of the nodes in u's graph neighborhood and generates a message based on that information. Them, the UPDATE function combines the message $\mathbf{m}_{\mathcal{N}(u)}^{(k)}$ with the previous embedding $\mathbf{h}_u^{(k-1)}$ of node u to generate the updated embedding $\mathbf{h}_u^{(k)}$ [9]. The first embedding is the input features for all nodes. The final layer output defines the embedding for each node after K iterations of GNN message passing [9].

$$\mathbf{z}_u = \mathbf{h}_u^{(K)}, \forall u \in \mathcal{V} \tag{2.14}$$

After K iterations, every node embedding will include information about the node's k-hop neighborhood. This information can be either structural information about the graph (e.g., degrees of all the nodes in u's k-hop neighborhood), or feature-based (all the features in each node k-hop neighborhood) [9].

Furthermore, the Message passing with Self-loop eliminates the need to specify an explicit update function because the aggregation method defines the update implicitly. This simplifies message passing and reduces overfitting, but drastically limits the expressivity of the GNN, as information from neighboring nodes cannot be distinguished from information from the node itself [9]. The most common disadvantage of GNNs is the over-smoothing issue [9] which appears after several message passing iterations, when the representations of all nodes in the graph start becoming too similar to one another. A simple solution to this problem is to utilize vector concatenations or skip connections, which attempts to retain data from earlier rounds of message transmission throughout the update stage and preserve more the original node-level information during message passing (Eq. 2.15).

$$\text{UPDATE}_{\text{concat}}\left(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}\right) = \left[\text{UPDATE}_{\text{base}}\left(\mathbf{h}_u, \mathbf{m}_{\mathcal{N}(u)}\right) \oplus \mathbf{h}_u\right] \tag{2.15}$$

Where the output of the base update function is concatenated with the node's previous-layer representation. In general, concatenation and residual connections are basic tactics that can help reduce over-smoothing in GNNs while enhancing optimization numerical stability. Using residual connections in GNNs, as in convolutional neural networks (CNNs) [29], can help train more complex models. In fact, these strategies work best for node classification tasks with deep GNNs (2-5 layers) and tasks that demonstrate homophily, i.e., where each node's prediction is significantly tied to its local neighborhood's attributes [9].

There are several techniques that improve *AGGREAGATE's* method performance. The first category is the pooling-based methods. Firstly, Set Pooling proposed by Zaheer et al. in 2017 [30] shows that an aggregation function with the following form is a universal set function approximator:

$$\mathbf{m}_{\mathcal{N}(u)} = \text{MLP}_\theta\left(\sum_{v \in N(u)} \text{MLP}_\phi\left(\mathbf{h}_v\right)\right) \tag{2.16}$$

whereas usually we use MLP_θ to denote an arbitrarily deep multilayer perceptron parameterized by some trainable parameters θ. This demonstrates that any permutation-invariant function mapping a collection of embeddings to a single embedding may be estimated to any degree of precision. The second pooling technique is named Janossy, which is also a powerful technique [31]. Janossy pooling rather than using a permutation-invariant reduction (e.g., sum or mean), employs a permutation-sensitive function and averaging the result over a large

number of potential permutations. The Janossy pooling method performs neighborhood aggregation as follows:

$$\mathbf{m}_{\mathcal{N}(u)} = \text{MLP}_\theta \left(\frac{1}{|\Pi|} \sum_{\pi \in \Pi} \rho_\phi \left(\mathbf{h}_{v_1}, \mathbf{h}_{v_2}, \ldots, \mathbf{h}_{v_{|\mathcal{N}(u)|}} \right)_{\pi_i} \right) \tag{2.17}$$

Where P_i is a set of permutations, ρ_ϕ is a permutation-sensitive function (an LSTM [32]), and $\pi_i \in \Pi$ takes an unordered set of neighbor embeddings and arranges them according to an arbitrary ordering.

Aside from generic aggregation methods, assigning attention weight to each neighbor is a common technique for enhancing the aggregation layer in GNNs [33]. The *Graph Attention Network* (GAT) was the first attention GNN introduced by Velickovic et al [34], which uses attention weights to define a weighted sum of the neighbors:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \left(\frac{\exp\left(\mathbf{a}^\top \left[\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_v \right] \right)}{\sum_{v' \in \mathcal{N}(u)} \exp\left(\mathbf{a}^\top \left[\mathbf{W}\mathbf{h}_u \oplus \mathbf{W}\mathbf{h}_{v'} \right] \right)} \right) \mathbf{h}_v \tag{2.18}$$

where a is a trainable attention vector, W is a trainable matrix, and $\bigoplus$ denotes the concatenation operation. The part of the equation included in "()" can be represented as $\alpha_{(u,v)}$ which is the attention on neighbor $v \in N(u)$ while aggregating information at node u [9]. Also, it is possible to add multiple attention "heads" in a transformer architecture [35]. Using separately specified attention layers, this technique computes K different attention weights $\alpha_{u,v,k}$. The collected messages are then transformed and merged in the aggregation stage, using a linear projection followed by a concatenation operation: $\mathbf{m}_{\mathcal{N}(u)} = [\mathbf{a}_1 \oplus \mathbf{a}_2 \oplus \ldots \oplus \mathbf{a}_K]$. Transformers are popular architecture in the domain of natural language processing, with the BERT [36] being the most popular. If the GNN is given a fully connected graph as input, the basic transformer layer is precisely equal to a GNN layer with multi-headed attention.

Another way to enhance the gating method is by using recurrent neural networks (RNNs). In particular, one way to think about the GNN message transmission method is that the aggregation function receives observations from neighbors and then uses them to update each node's hidden state [9], and directly apply methods used to update the hidden state of RNN architecture [37]:

$$\mathbf{h}_u^{(k)} = \text{GRU}\left(\mathbf{h}_u^{(k-1)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)} \right) \tag{2.19}$$

where GRU is the update equation of the gated recurrent unit (GRU) cell [38]. Notably, the parameters used to update the RNN are always shared between nodes. Therefore, these gated updates are extremely

successful at enabling deep GNN architectures (i.e., more than ten layers) and avoiding over-smoothing [9]. For the final layer of the GNN, the simplest way to enhance the quality of the final node representations is to use the representations at each layer of message transmission rather than relying just on the last layer output [9]. Thus, the final node representations z_u is:

$$\mathbf{z}_u = f_{\text{JK}}\left(\mathbf{h}_u^{(0)} \oplus \mathbf{h}_u^{(1)} \oplus \ldots \oplus \mathbf{h}_u^{(K)}\right) \tag{2.20}$$

where f_{JK} is an arbitrary differentiable function. This method is known as adding jumping knowledge (JK) connections [39]. In several cases, the function f_{JK} may be easily written as the identity function, which simply concatenates the node embeddings from each layer.

2.4.2 Graph Convolution Networks

The *graph convolutional network (GCN)*, one of the most common baseline graph neural network models, uses symmetric-normalized aggregation, which was employed by Kipf and Welling in 2016 [40], and self-loop update.

One GCN-based state-of-the-art model is the GraphSAGE (SAmple and aggreGatE) proposed by Hamilton et al. [41]. GCN use its adjacency matrix as convolution operation while GraphSAGE is differentiated because it uses Aggregation function as convolution operation. The Figure 2.1 depicts the three steps for the embedding generation of the GraphSAGE mode. To generate an embedding for a node, this method first collect the node's k-hop neighborhood (occasionally sub-sampling the full neighborhood for efficiency). In the next step, this methods aggregate the attributes of node's neighbors, using neural network aggregators. This aggregated neighborhood information is used to generate an embedding, which is then fed to the decoder.

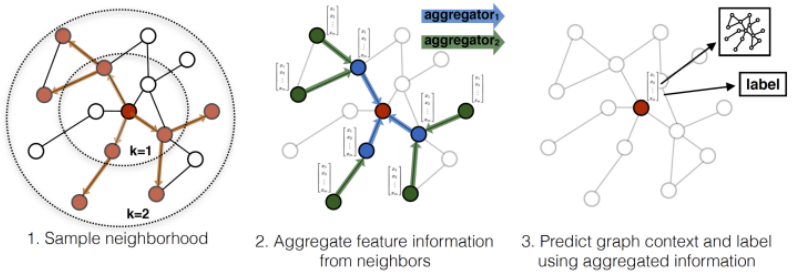


FIGURE 2.1: GraphSAGE Overview [41]

Furthermore, GNN are a bit different in the case of multi-relational or heterogeneous graphs. The first suggested technique is referred to

as the Relational Graph Convolutional Network (RGCN) method [42], which can use multiple relation types by specifying a separate transformation matrix per relation type:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{\tau \in \mathcal{R}} \sum_{v \in \mathcal{N}_\tau(u)} \frac{\mathbf{W}_\tau \mathbf{h}_v}{f_n(\mathcal{N}(u), \mathcal{N}(v))} \tag{2.21}$$

where f_n is a normalization function that can depend on both the neighborhood of the node u as well as the neighbor v being aggregated over [9]. One disadvantage of the naive RGCN technique is the dramatic rise in the number of parameters, since each relation type now has its own trainable matrix. Thus, a solution to this disadvantage is the parameter sharing RGCN technique can learn both an embedding for each relation and a common tensor for all relations, as Schlichtkrull et al. [42] proposed by using a parameter sharing with basis matrices. The relational GNN technique, in which each connection has its aggregation parameter, is suitable for multi-relational graphs and instances with discrete edge features. This can be accomplished by utilizing these features in attention or by concatenating them with neighbor embeddings during message transit by defining a new aggregation function as [43]:

$$\mathbf{m}_{\mathcal{N}(u)} = \text{AGGREGATE}_{\text{base}} \left(\left\{ \mathbf{h}_v \oplus \mathbf{e}_{(u,\tau,v)}, \forall v \in \mathcal{N}(u) \right\} \right) \tag{2.22}$$

where $e_(u, \tau, v)$ denotes an arbitrary vector-valued feature for the edge (u, τ, v).

2.4.3 Graph Pooling

The neural message-passing approach produces a set of node embeddings, so it is time to describe methods that learn an embedding for the entire graph G. In this case, we want to create a pooling function f_p that maps a collection of node embeddings $z_1, \ldots, z_{|V|}$ to a complete graph embedding z_G [9]. Two techniques are most commonly applied for learning graph-level embeddings using set pooling (sets of neighbor embeddings):

- Take a sum (or mean) of the node embeddings. For applications involving tiny graphs, pooling based on the sum or mean of the node embeddings is often adequate:

$$\mathbf{z}_{\mathcal{G}} = \frac{\sum_{v \in \mathcal{V}} \mathbf{z}_u}{f_n(|\mathcal{V}|)} \tag{2.23}$$

  where f_n is some normalizing function (e.g., the identity function).

- Use a combination of LSTMs and attention to pool the node embeddings [44]. This pooling technique iterates through a sequence of attention-based aggregations specified by the following set of equations for t = 1,..., T steps:

$$\begin{aligned} \mathbf{q}_t &= \text{LSTM}\left(\mathbf{o}_{t-1}, \mathbf{q}_{t-1}\right) \\ e_{v,t} &= f_a\left(\mathbf{z}_v, \mathbf{q}_t\right), \forall v \in \mathcal{V} \\ a_{v,t} &= \frac{\exp\left(e_{v,i}\right)}{\sum_{u \in \mathcal{V}} \exp\left(e_{u,t}\right)}, \forall v \in \mathcal{V}, \\ \mathbf{o}_t &= \sum_{v \in \mathcal{V}} a_{v,t} \mathbf{z}_v \end{aligned} \tag{2.24}$$

  Where q_t vector is a query vector for the attention at each iteration t, $e_{v,t}$ computes an attention score over each node using an attention function, which is then normalized, $\mathbf{o}_t$ is a weighted sum of the node embeddings. After T iterations, the output is the concatenation of all $\mathbf{o}_t$.

The only drawback of the graph pooling methods is that they do not exploit the graph's structure. To face this problem graph coarsening (clustering) methods were developed which pool the node representations. These techniques require the existence of a clustering function that maps all nodes in the graph to an assignment across c clusters. Regardless of the technique used to produce the cluster assignment matrix S, the basic principle of graph coarsening techniques is to coarsen the graph using this matrix, by computing a new adjacency matrix and set of features. Thus, the new adjacency matrix will reflect the edges (i.e., the strength of association) between the clusters in the graph, and the new feature matrix will represent the aggregated embeddings for all nodes allocated to each cluster [9]. After that, the final representation of the graph is calculated via set pooling over the nodes' embeddings in a sufficiently coarsened graph.

2.5 Practical Implementation

In this section, we examine how some of the above techniques may be expressed into specific loss functions for GNNs and how GNNs can be pre-trained unsupervisedly to increase their performance.

Pre-training strategies have evolved into a de facto norm in deep learning [45]. Neighborhood reconstruction loss may be an effective method for enhancing performance on a downstream classification assignment and equally strong compared to randomly initialized GNN [46]. This conclusion may be explained by the fact that GNN message passing already encodes neighborhood information. Due to the nature of message passing, neighboring nodes in a network tend to have

similar embeddings, making a neighborhood reconstruction loss redundant [9]. Therefore, Veliscovic et al. [46] proposed Deep Graph Informax, which involves maximizing the mutual information between node embeddings z_u and graph embeddings z_G. This example comes under the umbrella of the unsupervised objectives [47] and involves training GNNs to maximize mutual information between multiple levels of representation or to discriminate between genuine and corrupted pairs of embeddings.

2.5.1 Efficiency Concerns and Parameter Sharing

All the techniques we described above can be computationally inefficient. Thus, it is important in this section to provide some methods that can implement GNNs efficiently. To reduce the complexity of the message passing in Equation 2.12, a simple method can be using the message passing operations on sparse matrix multiplications [9], as:

$$\mathbf{H}^{(k)} = \sigma\left(\mathbf{A}\mathbf{H}^{(k-1)}\mathbf{W}^{(k)}_{\text{neigh}} + \mathbf{H}^{(k-1)}\mathbf{W}^{(k)}_{\text{self}}\right) \tag{2.25}$$

where $\mathbf{H}^{(k)}$ is a matrix containing the layer-k embeddings of all the nodes in the graph. With this improvement, the computation of the embedding for each node u happens only once, but that increase the memory needs, because it operates on the entire graph and all node features simultaneously. Thus, to limit memory footprint, mini-batch training is required, and it can work with a subset of nodes during message passing. The limit here is when a node is deleted then its edges are deleted too (i.e., modify the adjacency matrix) [9]. One strategy for avoiding this, is proposed by Hamilton et al. [9] and the basic concept is to choose a group of target nodes for a batch and then recursively sample their neighbors in order to maintain the graph's connection. GNNs are usually regularized using popular machine learning methods such as L2 regularization, dropout and layer normalization. But there are two more graph-oriented methods that can be used for GNNs, such as:

- Parameter Sharing Across Layers. The fundamental concept is to utilize the same parameters in all of the GNN's AGGREGATE and UPDATE functions. Generally, this strategy is most successful when used in conjunction with gated update functions for GNNs with more than six layers [48].

- Edge Dropout. During training, it randomly deletes (or **mask**) edges from the adjacency matrix, with the intention of making the GNN less prone to overfitting and more resilient to adjacency matrix noise. This method has been especially effective in the

application of GNNs to knowledge graphs [14] and it was a critical strategy in the pioneering work on graph attention networks (GATs) [34].

2.6 Graph Generation

Graph generation aims to create realistic graph topologies and it is similar to the graph embedding problem. In graph generation, we want our model to produce a graph G instead of a graph structure $G = (V, E)$. Of course, constructing an arbitrary graph isn't difficult. Graph generation traditionally involves providing a generative method that describes how the graph's edges are formed.

This generative method may be used to describe the probability or likelihood $P(A_{[u,v]} = 1)$ of an edge between two nodes u and v. The objective is to create a tractable generating process that can build graphs with non-trivial properties or characteristics [9]. The basic and simplest generative model is the Erdos-Renyi (ER) model [49] which is based on controlling the density of the graph. Improving this method, Stochastic block models were proposed [50], which seek to generate graphs with community structure. The idea is that every node has a probability of belonging to a block, and edge probabilities are then specified by a block-to-block probability matrix which gives the probability of an edge occurring between a node in block C_i and a node in block C_j [9]. The

Unlike the SBM model which generates graphs with community structures, the preferential attachment (PA) attempts to capture this characteristic property of real-world degree distributions [51]. The PA model is based on the premise that many real-world graphs have power law degree distributions. Power law distributions are heavy tailed, which means that the probability distribution approaches zero for very large values. Power law degree distributions intuitively convey the fact that real-world networks contain both a large number of nodes with small degrees and a small number of nodes with extremely high degrees. They key idea for the PA to generate graphs with power law distribution is for the model to connect new nodes to existing nodes with a probability proportional to the degrees of the existing nodes. This indicates that as the network expands, high degree nodes will tend to gather an increasing number of neighbors. Moreover, this method differentiates from the two above because generation process is autoregressive, meaning it specifies the edges iterative in each step.

2.6.1 Deep Learning Generative Methods

Despite the previous methods could be used to give us insight into how certain graph structures might arise in the real world, they cannot learn a generative model from data. Therefore, approaches that learn a generative model of graphs based on a set of training graphs are needed, such as variational autoencoders (VAEs), generative adversarial networks (GANs), and autoregressive models [9].

2.6.1.1 Varioational Autoencoders

The objective of the VAEs is to train a probabilistic decoder model $p_\theta(A|Z)$ from which realistic graphs (i.e., adjacency matrices) $\hat{\mathbf{A}} \sim p_\theta(\mathbf{A} \mid \mathbf{Z})$ may be sampled using a latent variable Z. In other words, it learns a conditional distribution over adjacency matrices [9]. Its training phase is a combination of a probabilistic decoder with a probabilistic encoder model. The encoder converts an input graph G to a posterior distribution over the latent variable Z [9]. The objective is to train the encoder and decoder together such that the decoder can rebuild training graphs from an encoder latent variable. The decoder can then construct new graphs by sampling latent variables. [9].

In VAE the key idea is that the encoder creates *latent representations* for each node in the graph. The decoder then accepts pairs of embeddings as input and utilizes them to forecast the probability of an edge forming between two nodes [52]. In more details, the encoder model is based on two GNNs two generate a mean embedding parameter for each node in the input graph, and a variance parameter which is a matrix similarly that specifies the log-variance for the latent embedding of each node [9].

When the model acquires the matrix of a sampled node embeddings $\mathbf{Z} \in \mathbb{R}^{|V| \times d}$, the decoder predicts the likelihood of all the edges in the graph. One popular method for the decoder is to employ a simple dot-product, such as:

$$p_\theta\left(\mathbf{A}[u,v] = 1 \mid \mathbf{z}_u, \mathbf{z}_v\right) = \sigma\left(\mathbf{z}_u^\top \mathbf{z}_v\right) \tag{2.26}$$

where σ is used to denote the sigmoid function. To calculate the reconstruction loss, we simply assume independence between edges and define the posterior$p_\theta(\mathcal{G} \mid \mathbf{Z})$ over the full graph as follows :

$$p_\theta(\mathcal{G} \mid \mathbf{Z}) = \prod_{(u,v) \in \mathcal{V}^2} p_\theta\left(\mathbf{A}[u,v] = 1 \mid \mathbf{z}_u, \mathbf{z}_v\right) \tag{2.27}$$

which corresponds to a binary cross-entropy loss over the edge probabilities [9]. A major limitation of this methods is that without a training graph, the model cannot produce non-trivial graph topologies.

Graph-level Latents Encoder-Decoder approach uses graph-level latent representations instead of node-level representations [53]. The encoder uses two separate GNNs to parameterize the mean and variance of a posterior normal distribution over latent variables [9]. The difference is that the parameters are generated for a single graph-level embedding, while previously were defined for each node.

The decoder of the graph-level VAE works similarly with the node-level VAE. It uses an MLP to map the latent vector z_G to a matrix of edge probabilities [9], $\tilde{\mathbf{A}} = \sigma\left(\text{MLP}\left(\mathbf{z}_{\mathcal{G}}\right)\right)$, where the sigmoid function σ is used to guarantee entries in [0, 1]. Thus, the loss function is defined similarly to the node-level case, as:

$$p_\theta\left(\mathcal{G} \mid \mathbf{z}_{\mathcal{G}}\right) = \prod_{(u,v)\in\mathcal{V}} \tilde{\mathbf{A}}[u,v]\mathbf{A}[u,v] + (1-\tilde{\mathbf{A}}[u,v])(1-\mathbf{A}[u,v]) \quad (2.28)$$

where A denotes the true adjacency matrix of graph G and $\tilde{A}$ is the predicted matrix of edge probabilities. Using an MLP decoder requires a set number of nodes. Generally, a maximum number of nodes is assumed and a masking strategy is used.

In the framework of VAEs in biomedical science, Jin et al. [54] proposed an advanced variant of the graph-level VAE approach that leverages knowledge about known molecular motifs. Moreover, this approach is divided into two sub-tasks:

1. Learning to represent molecules in a continuous manner that enables prediction and optimization of their characteristics (encoding).

2. Learning to convert an optimized continuous representation back to a molecular graph with enhanced properties (decoding)

2.6.1.2 Adversarial Approaches

In this category the most popular method is the generative adversial netowrks (GANs) [45]. A generic GAN-based generative model works as follows. First, it is necessary to define $g_\theta : \mathbb{R}^d \longrightarrow X$ as a trainable generator network. Given a random seed $\mathbf{z} \in \mathbb{R}^d$, this generator network is trained to create realistic (but fraudulent) data samples $\tilde{x} \in X$. Also, it is required to define a discriminator network $d_\phi : X \longrightarrow [0,1]$. The discriminator's purpose is to discriminate between genuine data samples $x \in X$ and generator samples $\tilde{x} \in X$. To train a GAN, both the generator and discriminator are optimized jointly in an adversarial game [9]:

$$\min_\theta \max_\phi \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}(\mathbf{x})}\left[\log\left(1-d_\phi(\mathbf{x})\right)\right] + \mathbb{E}_{\mathbf{z}\sim p_{\text{seed}}(\mathbf{z})}\left[\log\left(d_\phi\left(g_\theta(\mathbf{z})\right)\right)\right] \quad (2.29)$$

where $p_{data}(x)$ is the empirical distribution of real data samples (e.g., a uniform sample over a training set) and $p_{seed}(z)$ is the random seed distribution (e.g., a standard multivariate normal distribution). In other words, the generator tries to reduce the discriminator's discriminating power, while the discriminator tries to increase its ability to recognize fake samples [55]. In the framework of graph generation the GAN-based method is similar with the VAE [56]. Thus, in order to construct an edge probability matrix from a seed vector z, an MLP is required to be normalized using a sigmoid function. Therefore, it is possible to use the edge probability matrix to create a discrete adjacency matrix by selecting independent Bernoulli variables for each edge, while the discriminator can be based on a GNN graph classification model [9].

Chapter 3

Drug Target Interaction Prediction

Having discussed about graph representation learning in the previous chapter 2, this chapter is going to provide the problem definition of the Drug Target Interaction. Firstly, this chapter describes why is important to use computational methods for drug discovery, instead of lab experiments. Then, network-based methods of finding new drugs is described. Finally, description of related work in DTI field is provided, focusing the field of network-based approaches.

3.1 Introduction to Drug Target Interaction

Drug Target Interaction (DTI) refers to the binding of a drug to a target location that results in a change in its behavior/function [57]. A drug is a term from pharmacology to define a chemical substance used in the treatment, cure, prevention, or diagnosis of disease or used to otherwise enhance physical or mental well-being [58]. A drug's target is a molecule in the body, usually a protein, that is intrinsically associated with a particular disease process and that could be addressed by a drug to produce a desired therapeutic effect [59]. DTI prediction is essential for the drug discovery process aiming to find novel therapeutic molecules for biological targets. Moreover, the drug's various target interactions are also responsible for its side effects. As a result, DTI prediction can aid in drug development, repositioning, and side effect prediction, as well as potentially lowering drug costs. The interaction process happens when the attached drug reacts with the target to create a positive or negative change, because the chemical component of the drug forms transient bonds with the target molecule [57], as shown in Figure 3.1. Rather than receptors, some drugs target enzymes, which are able to regulate the pace of chemical reactions. Inhibitors and inducers are two types of drugs that target enzymes. Enzyme induction usually increases glucuronyl transferase activity, and thus enhances drug conjugation; it may also decrease intracellular haem and enhance porphyrin synthesis. For example, drugs that

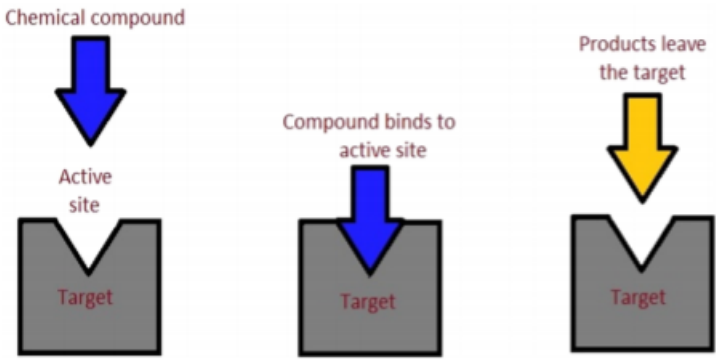


FIGURE 3.1: Drug Target Interaction process [57]

inhibit cytochrome P450 may increase plasma concentrations of other concurrently used drugs, resulting in drug interactions [60]. On the other hand, enzyme inhibitors are compounds which modify the catalytic properties of the enzyme and, therefore, slow down the reaction rate, or in some cases, even stop the catalysis. Such inhibitors work by blocking or distorting the active site [61]. In other words, inhibitors reduce the activity of enzymes, whereas inducers enhance it.

The basic way to discover a drug is through wet-lab experiments (in-vitro) by using different techniques of classical and reverse pharmacology [62]. Even though is the most reliable way to discover new drugs, it is time-consuming (e.g., the US Food and Drug Administration approves only about 20 new drugs in a year [63]) and very expensive because every drug needs to undergo several trials in order for the customer to be able to buy it. The cost of identifying each new molecular entity (NME) is approximately $1.8 billion, while the time it takes to reach the patients is close to ten years [64]. Thus, it is needed a way to reduce those costs by reducing the number of wet-lab experiments, such as of using computation methods.

Additionally, polypharmacology (one drug, several targets) indicates that a drug that may have a therapeutic impact on one target may have a side effect on another target. The term "drug side effects" refers to the undesirable effects that a medication may have and constitute a significant issue in the field of drug development. Currently, drug failures are the fourth-largest cause of death in the US [65], which means that early prediction of drug side effects at the clinical stage will aid in streamlining the time-consuming and costly process of drug testing [66].

On the other hand, there are millions of small molecule compounds that have still unknown interaction profiles (e.g., PubChem database [67]). As a result, drug re-purposing is seen as an efficient method for treating an illness using an already developed drug. This is possible as any given chemical may interact with numerous biological targets and frequently does. Recognizing this, scientists have embraced

polypharmacology as a therapeutic paradigm in recent years, replacing the conventional dogma of "one drug, one target, one disease." This approach provides a number of advantages. To begin, an already-approved drug has undergone rigorous clinical studies prior to commercialization. As a result, the drug's safety has already been verified. All the above made the necessity for more reliable computational methods (in-silico) for interactions between drugs and targets [68].

Nowadays, there are three main computational method categories for Drug Target Interaction prediction: ligand-based approaches, docking approaches, and chemogenomic approaches [68].

1. Ligand: This category is based on the idea that similar molecules that share similar properties, usually bind to similar protein targets [69]. Therefore, there are some approaches that predict the interactions based on the similarities between protein ligands [70]. An example of this technique is the Quantitative Structure-Activity Relationships (QSAR) which compares a prospective ligand to the set of all known ligands to infer its binding capacity [71]. This method, however, has a number of drawbacks. Because the protein sequences are not utilized in the prediction, the potential new interactions are restricted to those between known ligands and protein families. Although this category is useful for DTI prediction, it is unreliable when the number of known binding ligands per protein is small [72].

2. Docking: This category needs the structures of a drug and a protein to run a simulation to predict if they will interact [73]–[75]. An important limitation is that there are some proteins for which the 3D structure is still challenging to infer (e.g., membrane proteins) and therefore, docking methods cannot be applied [76], [77].

3. Chemogenomic: The third category uses information from both the drug and target at the same time. More precisely, in order to find possible interactions, the drugs space and the genomic space of proteins have to be unified in a common subspace [57]. This category of methods does not have the previous disadvantages because it can leverage abundant biological data to perform predictions. For example, the genomic sequences and the chemical structure graphs have already been used in public databases [78].

In the Ezzat et al. survey [68], the authors focused more on the third category of DTI prediction methods and they further divided it into five subcategories.

- Neighborhood models: This type of models are based on the nearest neighbor of the interaction profile for the drug (or target).

- Bipartite local models: These kind of methods create final prediction scores for specified drug–target pairs by performing two sets of predictions independently, one from the drug side and one from the target side.
- Network diffusion models: These approaches study the impact propagation in drug–target networks using graph-based algorithms (e.g. Random Walk) to identify new DTIs.
- Matrix factorization models: Initially, the DTI matrix is used to learn the latent feature matrices for drugs and targets, and then these two latent feature matrices are multiplied together to rebuild the interaction matrix generating predictions.
- Feature-based classification models: The feature-based techniques identify the interactions between drug-target pairs by determining characteristics that are more discriminative than the baseline features. The inputs to these techniques are feature vectors that may be generated by combining the feature vectors of drug-target pairs. Consequently, in order to generate new interactions, a variety of machine learning models such as random forest or Support Vector Machines (SVM) are then applied. Most commonly the problem is formulated as a binary classification task, where the drug-target vector is the data and the labels are interaction and non-interaction. However there are some related works that use regression to approach the problem of DTI prediction, by representing the interaction between a drug and a target as a continuous number between 0 and 1 and considering and an interaction any number above a certain threshold.

In this thesis we will focus on the third category, which is the Network diffusion models, and exploit Graph Neural Networks and graph theory strategies for predicting new connections; the network diffusion technique is the most prevalent in this area.

3.2 Related Work

In this section we will describe related work that has been published on conferences or journals. The related work will focus solely of works which are using network-based methods.

3.2.1 DTI with Graph Representation Learning

First of all, Wang et al. in their publication [79], described that the drug-target heterogeneous graph has two kinds of nodes: drug nodes and target nodes. Let us assume that $R = R_1, R_2, \ldots, R_n$ denote the n

drug nodes, and $T = T_1, T_2, \ldots, T_m$ denote the m target nodes. In this case, the heterogeneous network augments the basic bipartite one by adding, between all pairs of drugs or targets, edges whose weights correspond to pairwise similarities. So, now the target prediction problem can be transformed into a drug-target edge prediction problem on the constructed drug-target graph. Thus, the network diffusion in this method is done using the following equation:

$$W_{rt}^{i+1} = \alpha W_{rr} \times W_{rt}^{i} \times W_{tt} + (1 - \alpha) W_{rt}^{0} \tag{3.1}$$

In this formula, $W_{rt}^{0} = W_{rt}$, represents the initial interactions between drugs and targets, a is a decay factor with value between 0 and 1. In each iteration, the original drug-target interactions will contribute to the newly constructed interactions, and the contribution is controlled by the scale factor $1 - a$. By iteratively using this formula, the model will eventually include all the possible paths connecting them in the heterogeneous graph. The W_{rr} and W_{tt} are the weights of the drugs and targets (drug-drug and target-target) and they have to converge. To assure the convergence, they normalized those parameters beforehand using the following equations:

$$w\left(r_i, r_j\right) = \frac{w\left(r_i, r_j\right)}{\sqrt{\sum_{k=1}^{n} w\left(r_i, r_k\right) \sum_{k=1}^{n} w\left(r_k, r_j\right)}}, \tag{3.2}$$

$$w\left(t_i, t_j\right) = \frac{w\left(t_i, t_j\right)}{\sqrt{\sum_{k=1}^{m} w\left(t_i, t_k\right) \sum_{k=1}^{m} w\left(t_k, t_j\right)}} \tag{3.3}$$

Another more advanced technique that uses Random Walks with Restart besides a heterogeneous network is the method *"Network-based Random Walk with Restart on the Heterogeneous network (NRWRH)"* which is described in Chen et al. [80]. To perform Random Walk, NRWRH uses the transition matrix:

$$M = \begin{bmatrix} M_{\mathrm{TT}} & M_{\mathrm{TD}} \\ M_{\mathrm{DT}} & M_{\mathrm{DD}} \end{bmatrix} \tag{3.4}$$

where M_{TD} and M_{DT} are transition matrices from targets to drugs and from drugs to targets, and M_{DD} and M_{TT} are the intertransition matrix indicating the probability from one target (drug) to the other target (drug) in the random walk, respectively. Furthermore, the equation that assumes the i-th element that holds the p_t a vector probability that finds the random walker at node i at step t is:

$$p_{t+1} = (1 - r) M^{T} p_t + r p_0 \tag{3.5}$$

where the parameter r is the restart probability, the $p_0 \in \mathbb{R}^{(n+m)\times(n+m)}$ is the adjacency matrix between the nodes (including the n drugs and m targets in both rows and columns). Therefore, by running the above equation until convergence, the matrix p will result the final prediction score.

Probabilistic Soft Logic (PSL) [81] also uses a heterogeneous network and a syntax based on first-order logic as a templating language for probabilistic models over continuous variables. PSL uses the Lukasiewicz-norm and co-norm to provide relaxations of probabilistic soft logic to perform prediction (i.e., $AND(\wedge), OR(\vee)$, and $NOT(\neg)$). The main idea of this technique is that the similarity among drugs may imply similar interactions with targets, and similarity among targets may imply similar interactions with drugs. To ascertain if a drug and target interact, triad and tetrad relations (i.e. pathways of length three and four, respectively) between the drug and target are identified and utilized to predict the possible interaction. This method assumes that similar targets tend to interact with the same drug and that similar drugs tend to interact with the same target, so it is based on triangles or triads between drugs and targets. Triad rules take the form of:

$$\begin{aligned} \text{SimilarTarget}_{\beta}(T_1, T_2) \wedge \text{Interacts}(D, T_2) \\ \rightarrow \text{Interacts}(D, T_1) \\ \text{SimilarDrug}_{\alpha}(D_1, D_2) \wedge \text{Interacts}(D_2, T) \\ \rightarrow \text{Interacts}(D_1, T) \end{aligned} \tag{3.6}$$

where T denotes a target, D indicates a drug, the predicate $SimilarTarget_{\beta}$ represents a specific target-target similarity metric.

In addition to triads, this method also implements more complex templates for reasoning about both drug and target similarities to predict interactions. The equation for this rule is:

$$\begin{aligned} \text{SimilarDrug}_{\alpha}(D_1, D_2) \wedge \text{Similartarget}_{\beta}(T_1, T_2) \\ \wedge \text{Interacts}(D_2, T_2) \rightarrow \text{Interacts}(D_1, T_1) \end{aligned} \tag{3.7}$$

where α and β are drug-drug or target-target similarity measures. To predict new interactions, the aforementioned rules are applied wherever they are applicable on the DTI network (i.e., each rule is applied to the network's associated relations). Additionally, in order to avoid investigating all possible triad and tetrad relationships, a blocking technique is used in advance, in which edges between pairs of drugs or targets (which correspond to pairwise similarities) are removed from the network if their weights (i.e. similarity values) are less than a user-defined lower bound.

Determine All Simple Paths, Find Interactions (DASPfind) is a method

proposed by the university of KAUST [82]. This method uses a heterogeneous graph consisting of three subgraphs connected to each other, which are: drug–drug similarity, protein–protein similarity, and known drug– protein interactions. It predicts an interaction between a drug d and a target t by finding all simple paths (i.e. that have no cycles) connecting them. For the found simple paths, each path p has its score s_p computed by multiplying the weights on its edges. The assumption of this method is that a drug and a protein have a higher probability to interact if there are more paths connecting them as the paths represent confident relationships between the nodes. Moreover, longer paths should contribute less to the aggregated score, indicating the probability of an interaction occurring between the path's end nodes. The equation of how the aggregation of the score happens on these different paths is:

$$\text{score} = \sum_{\text{p=1}}^{\text{n}} \left(\prod \text{p}_\text{w}\right)^{\alpha \times \text{len(p)}} \tag{3.8}$$

where $p = p1, p2, \ldots, p_n$ is the set of the paths connecting a specific drug and a specific protein, p_w is the weights of the edges along an individual path, and $len(p)$ is the number of edges of that path; α is an adjustable decay parameter that was found empirically and it maximizes the average result across different benchmark datasets. Similar to PSL, the blocking process is employed here before prediction (i.e., edges with weights less than a specific threshold are eliminated).

KenDTI [83] is published by the Yu et al. in 2021. KenDTI is based on a heterogeneous network of drugs and targets and exploits their features, to integrate chemical and target space by an effective representation. It combines both network-based and classification-based methods. One model adopts network integration and recommendation algorithms. The other utilizes pre-trained embedding vectors for both drug molecules and target proteins, which are fed into convolutional neural networks to learn high-level representation for the classification. It transforms the drug SMILES and target proteins sequences into strings, using one-hot embedding vectors. Then, it uses the Word2Vec model from NLP to get the embedding vectors for each single character. Then, an 1D CNN was adopted to process the input embedding vectors of chemical molecules and proteins, and also to capture neighborhood information for each character at different scales. Therefore, the CNN produce a 512-D feature vector for each drug-target pair. The network-based predictor, firstly, performs a random walk with start on each heterogeneous network to extract topological relations of nodes, and then performs a matrix completion algorithm to discover the best projection from drug space onto target space. Finally, to combine the two predictors, they used a meta-classifier which generates the final prediction. The meta-classifier uses a softmax cross-entropy loss.

Liu et al. [84] introduced the GADTI which is a graph autoencoder approach for DTI prediction to discover potential interactions between drugs and targets using a heterogeneous network, which integrates diverse drug-related and target-related datasets. GADTI is divided into two parts, the encoder, and the decoder. The encoder is a GCN, which is used to aggregate first-order neighbor information to update node representation, followed by a Random Walk with Restart, which produces embeddings for nodes in the DTI graph. The GCN aggregates a node neighborhood to obtain information, while the first-order node neighborhood is grouped by the edge type. Then, the neighbor information from various groups is gathered and concatenated with the node's prior embedding vector and then passed into the neural network for the generation of a new embedding vector. This method is described by the following equation:

$$a_v^r = \sum_{u \in N_{(v)}} \frac{1}{c_r^v} \sigma\left(W_r^0 h_u^0 + b_r\right), h_v^* = MEAN\left(\left\{h_v^0\right\} \cup \{a_v^r : r \in R\}\right) \tag{3.9}$$

where a_r^v refers to the aggregated neighborhood information of v related to edge type r, $h_0^v \in \mathbb{R}_d$ is the initial embedding vector of v, d denotes the vector dimension. R is the set of edge types, $N_r(v)$ are the neighbors of v related to edge type r, σ is a non-linear activation function, and $W_r^0 \in \mathbb{R}^{d \times d}$ and $b_r \in \mathbb{R}_d$ are edge-type specific parameters matrix and bias terms used to aggregate neighborhood information, respectively. c_r^v is a normalization constant. $MEAN()$ $[Relu(x) = max(0, x)]$ is an element-wise mean operator, h_v^* is the updated embedding vector of v. Because GCN considers only the one-hop graph structure, a Random Walk with Restart is used to increase the receptive field of convolution, thereby achieving long-range message passing. The decoder reconstructs the original network from the embedding vectors. GADTI is using the DistMult [85] decoder and they apply an equation which is:

$$DTI_{re} = V_{drug}^T M_{DTI} V_{\text{protein}} \tag{3.10}$$

where V_{drug} and $V_{protein}$ are the matrices of drug embedding vectors and target embedding vectors, respectively, and M_{DTI} is the diagonal matrix used to reconstruct the DTI network. Finally, in the training procedure the optimization goal is to minimize the following equation:

$$\begin{aligned} L_{re} &= \sum_{r \in R} \sum_{i,j} \left(Q_{ij}\right)^2 + \lambda \sum_{w} w^2, Q \\ &= P\left(\text{ Networ } k^r_{\text{original}} - \text{ Networ } k^r_{\text{reconstruction}}\right) \end{aligned} \tag{3.11}$$

where $\sum_w w^2$ is the sum of the squares of all the weights, and λ is an adjustment coefficient, $Network_r$ original and $Network_r$ reconstruction are the original network with edge type r and the corresponding reconstructed network, respectively. P is a mask matrix and Q is a matrix that stores the difference between the predicted value and the ground truth in the training set.

Lim et al. [86] in 2019 proposed a distance-aware graph attention algorithm to differentiate various types of intermolecular interactions by extracting their graph feature directly from the $3D$ structural information on the protein - ligand binding pose. They proposed three different novelties. The first one was embedding the structural information on a protein-ligand complex, which is parted into two matrices of purely covalent interactions and both covalent interactions and non - covalent intermolecular interactions. The matrices can be formulated as follows:

$$\mathbf{A}^1_{ij} = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are connected by covalent bond or } i = j \\ 0 & \text{otherwise} \end{cases} \tag{3.12}$$

$$\mathbf{A}^2_{ij} = \begin{cases} \mathbf{A}^1_{ij} & \text{if } i, j \in \text{ protein atoms or } i, j \in \text{ ligand atoms} \\ e^{-\left(d_y - \mu\right)^2/\sigma} & \text{if } d_0 < 5 \text{ and } i \in \text{ ligand atoms and } j \\ & \in \text{ protein atoms, or if } d_y < 5A \text{ and } i \\ & \in \text{ protein atoms and } j \in \text{ ligand atoms} \\ 0 & \text{otherwise} \end{cases} \tag{3.13}$$

where d_{ij} is the distance between the i-th and j-th atoms, and μ and σ are learnable parameters. p indicates that intermolecular bonds are weaker than covalent bonds and that their strength decreases with increasing bond distance. The second novelty was the distance-aware graph attention mechanism and a gate augmentation algorithm. Its inputs are adjacency matrix (A) of node features, while its output is a set of new node features by using a learnable weight matrix. After, it obtains the attention coefficient, e_{ij}, which represents the importance of the j-th node feature to the ith node feature, as follows:

$$e_{ij} = x_i'^T \mathbf{E} x'_j + x_j'^T \mathbf{E} x'_i \tag{3.14}$$

where $\mathbf{E} \in \mathbb{R}^{F \times F}$ is also a learnable matrix. To manipulate the scale of the attention coefficients across nodes, the attention coefficient is normalized across neighbors.

Then each node feature is updated as a linear combination of its adjacent nodes' node features and the normalized attention coefficient.

The third novelty was the gate mechanism which directly delivers information from the previous node features to the next layer. Its output is a linear combination of the previous and the updated node features. All of the above were used to create the neural network which produces two new node features. After, the feature vectors of the ligand atoms are summed to form a vector reflecting the graph of the protein-ligand complex. Finally, multilayer perceptrons are used to x^{graph} to determine whether or not the protein-ligand complex or binding pose is active.

On the other hand, Chen et al. [87] proposed a solution that combines heterogeneous network features and NLP mechanisms to predict DTI. Because of the limited number of the atom and molecular bond types in the graph, they conducted a local breadth-first search (BFS) on a node to record the information on all adjacent nodes and edges of node i in the subgraph. Therefore, the initial node features x_i for their GNN model were extracted (e.g, atom type, charge, multiplicity, mass). After the aggregation of each node in the subgraph, the new embedding is inserted into the next GNN layer, by combining neighborhood information and previous embedding. To obtain the attention coefficients, to know which part of the molecular structure contributes more to the interaction between the protein and the compound, they used the following equation:

$$\text{coef}_{ij} = \text{LeakyReLU}\ \left(W_{\text{atten}} \cdot \left[h_i, h_j\right]\right) \tag{3.15}$$

where $W_{atten} \in \mathbb{R}^{F \times F'}$ is a learned matrix applied for nodes and LeakyReLU is the leaky rectified linear unit for the hidden features. Their final embedding of the compound was obtained by combining the normalized attention coefficients with the final compound embedding. For the NLP task, they used a trigram to split the amino acid sequence. Then, BERT was used to learn embedding vectors from the text composed of the protein "words" to produce a set of hidden vectors, which were inserted into a BiLSTM model. To emphasize the important subsequences, they adopted a sequential attention mechanism. Finally, the classifier layer concatenates the learned molecular embeddings from the GNN with the BiLSTM model's resultant protein vector representation. The final layer output is normalized using a sigmoid activation function. The final training purpose was to minimize the cross-entropy loss.

GanDTI [88] is based on the residual GNN and attention for ligand-protein binding prediction. The residual GNN has been used to train the compound graph features, which have been subsequently incorporated into the attention module to produce the compound-protein vectors for MLP processing. This approach was divided into three

main modules. The first module was the residual graph neural network which aggregates fingerprinted features to form the final compound features. Firstly, an adjacency matrix A is used to describe which atoms are bonded together and a message passing method that is widely adopted as:

$$c_i^{(l+1)} = U\left(c_i^{(l)}, \sum m\left(c_j^{(l)}\right)\right) \tag{3.16}$$

where $c_i^{(l)}$ denotes the compound molecule fingerprint features. It includes the information of node i and its edges at hidden layer l. U represents the update function. m represents the message passing function. A message-passing method is used to gather information into the graph-based weight matrix with a non-linear activation function. After 3 layers of iterations, initial fingerprint features were added to conclude the final compound features after the aggregation. The second module is the attention module that finds which part of the protein has a larger impact on the interaction. The compound and the protein are first transformed to another space in the following way:

$$\begin{aligned} h_c &= f\left(W_a c_e\right) \\ h_p &= f\left(W_a P\right) \end{aligned} \tag{3.17}$$

where h_c and h_p are the transformed compound features and protein features, f is the ReLU function, W_a is a learnable weight matrix. Then, they added dot-product attention using a *tahn* function to generate the score, that depicts the importance of the protein sequence and it can formulate a summarized protein feature vector as:

$$p_e = \sum_{i \in \text{ protein}} W_s \,\text{score}\left(c_e, P\right)^T h_p \tag{3.18}$$

where W_s is the scale weight of the protein feature. The last module is the prediction that generates the vector for the MLP processing using the concatenation of the compound vector and the protein feature vector. This model performs both binding affinity prediction, which minimizes the mean squared error, and interaction classification, which minimizes the cross-entropy loss.

DTI-GAT [89] is a deep neural network architecture that operates on graph-structured data with an attention mechanism. To begin, it transforms the position-specific scoring matrix (PSSM) and drug fingerprint into feature vectors for all target proteins and drugs. This matrix computes the average score of the residue in protein during the process of evolution, which is the mutation from amino acid type i to j. To retain the sequence information the pseudo-position specific scoring matrix (PsePSSM) for protein p_m is computed. Second, a DTI

graph is generated based on feature vector similarity. Each node represents a protein or a drug, and the nodes are connected by edges that reflect interactions between protein-drug, protein-protein, and drug-drug. Therefore, the PsePSSM and the fingerprint are transformed to the same dimension and weight matrices are applied for the nonlinearity ReLU to transform the feature dimension. The edge set, contains two different components, the interaction edge (1 or -1, referring to positive relation and negative relation of two nodes) and similarity edge which is based on the neighbors of the node, described as follow:

$$e_s(i,j) = \begin{cases} 1, & \text{if } \text{com_}n\left(h_i, h_j\right) > \theta \\ 0, & \text{if } \text{com_}n\left(h_i, h_j\right) \leq \theta \end{cases} \tag{3.19}$$

where com_n is the function to compute the common neighbor of two nodes and θ is the threshold. Then, embeddings for each protein and drug are generated. For that reason, firstly the input features transformed into a higher lever feature using a weight matrix and then a self-attention mechanism on node pair to compute attention weight, which indicates the importance of the node. After obtaining the normalized attention score, a message passing is used to compute a linear combination of the node features and output the final aggregated features of each node. The final decoder architecture predicts the interaction result. Finally, it assigns a binary class label to each identified drug-protein and its objective is to minimize the following binary cross-entropy loss (BCELoss).

LGDTI [90] is another proposal that predicts DTI based on large-scale graph representation learning and captures the local and global structural information of the graph. It implements a GCN model which has as input the structure of the graph and the characteristics of each node. The output results are three different outputs i) the results at the node level, ii) the results which are about the whole graph, and iii) the pooling information at the node level. The network communication rules of GCN are as follows:

$$f(X,A)^{l+1} = \sigma\left(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}X^l W^l\right) \tag{3.20}$$

where, $\widetilde{A} = A + I_{n+m}$ is the adjacency matrix added to the self-loop, $\widetilde{D}$ is represented as the degree matrix of $\widetilde{A}$. W is the weight of the randomly initialized network. σ represents the activation function of each layer of the neural network, here σ is a ReLU layer and, $f(X,A)$ is using the spatial method of GCN. Due to over-smoothing of the GCN, after each convolution, it adds node features for training. Furthermore, for graph embedding, it uses the DeepWalk algorithm. As for the Large-Scale Graph Representation, it needs a cycled undirected graph so, the adjacency matrix has 1 for connected nodes or 0 for non-connected nodes. Then, it maps data from complex

graph space to multi-dimensional space. Therefore, the first-order information is used to describe the local similarity between pairs of vertices in a graph and contains the attributes of nodes, by using the KL-divergence to calculate the distance as follow:

$$O_1 = -\sum_{(i,j)\in E} w_{ij} \log p_i\left(\boldsymbol{v}_i, \boldsymbol{v}_j\right) \tag{3.21}$$

where $p_1\left(\boldsymbol{v_i}, \boldsymbol{v_j}\right) = \frac{1}{1+\exp\left(-v_i^T \cdot v_j\right)}$ are the low-dimensional vector representation of the node v_i and v_j. W_ij is the edge weight between node i and j. The high-order information is learning the relationship between vertex v_i and the other vertices separately and contains the whole network information of the node, which is obtained by the DeepWalk embedding method.

AutoDTI++ [91] is an unsupervised DTI prediction method proposed by Sajad et al. Most of the previous proposals met sparse matrix problems, but this method faced this problem by adding information on drugs fingerprint to the interaction matrix. Thus, in pre-processing step the fingerprint-target matrix is created which is the multiplication of the fingerprint-drug matrix with the drug-target interaction matrix. The AutoDTI aims to take each partially observed drug or target as input, project it into a low-dimensional latent space and then reconstruct it in the output space to predict unknown interactions. The same formula applied for the two types of autoencoders (drug or target), which is the following:

$$\min_{\theta} \sum_{d=1}^{n} \|Y_d - nn\left(Y_d, \theta\right)\|_o^2 + \frac{\lambda}{2} \cdot \left(\|W\|_F^2 + \|V\|_F^2\right) \tag{3.22}$$

where it means that only the known interactions are considered and regularization strength $\lambda > 0$. To train the autoencoders three steps are required, i) Assign zero to unknown interactions in the edges of input layers, ii) back-propagated values in the output layers' edges are replaced with zero values; iii) a denoising loss is used to prioritize interaction prediction over reconstruction. AutoDTI++ is an expansion of AutoDTI. In the suggested AutoDTI++ approach, the fingerprint-target matrix is used as the input to the AutoDTI model rather than the drug-target interaction matrix.

A different aspect for the DTI problem proposed by Chu et al., that uses multi-label learning with community detection method and named it DTI-MLCD [92]. The first thing they did was to construct their dataset which is parted of 5 different databases and obtained drugs and targets. Python tools were used to represent drug features into three categories, molecular descriptors (MDs), molecular fingerprints (MFs), and semantic information features obtained from drug

SMILES using Word2vec. For Target representation they used three different feature types, i) Composition, Transition, and Distribution (CTD) using PROFEAT web server, ii) 1437 default protein descriptors generated by PROFEAT, and iii) the protein domain fingerprint (PDF) from the PFAM v31.0 database. This study divides the whole label space into many smaller label subspaces using the community detection approach. Thus, DTI-MLCD applies five data-driven community detection algorithms as label partitioning methods and assembles them into a multi-label learning method. Those five are the,

- Fast greedy algorithm (FGA). It is a modularity-based algorithm based on the greedy approach.
- Multi-level algorithm (MLA). Similar to the previous but with a different greedy approach for the modularity optimization.
- Label propagation algorithm (LPA). It is a diffusion-based technique based on semi-supervised graph learning that replicates the diffusion of flow on a network via label diffusion.
- Walk trap algorithm (WTA). It is a node similarity-based algorithm based on random walks.
- Infomap algorithm (IMA). It is a compression-based algorithm based on random walks.

Before the execution of community detection, it is required to construct the undirected weighted label co-occurrence graph, where the vertices represent the label set, the edges represent label pairings that appear at least once in the training label set, and the weight assigned to each edge is specified as the number of samples with both labels. Finally, they tested all community detection methods and multi-label classification methods with all datasets to find which combination has better performance.

Another proposal that used Graph Networks was the Xu et al. [93] from Wuhan University of Science and Technology, China, which uses a graphical representation of SMILES based on 5-layer Graph Isomorphic Network (GIN) and word vector matrices of amino acid sequences based on 4-layer one-dimensional convolution. Moreover, ProfVec (developed from Word2Vec) is used to encode the amino acid sequence as a word vector, where each atom is described by a feature vector and each atom v is calculated by dividing it with the sum of all atoms v. The output of the GIN is the input for the graph pooling layer which concludes to the same fully connected layer with the target feature. The target feature is retrieved using the convolutions, each of which is followed by a batch normalization layer and a ReLU activation layer. Then, a three-layer fully connected classifier is used to generate the prediction results. Each layer is followed by a ReLU activation layer

and a Dropout layer, and the output is obtained from a sigmoid layer, while the binary cross-entropy loss is calculated as follow:

$$l(x, y) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \cdot \log x_i + (1 - y_i) \cdot \log\left(1 - x_i\right)\right] \tag{3.23}$$

where n is the batch size x_i denotes the real label, either 0 or 1. y_i denotes the predicted score and is a value between 0 and 1.

Cheng et al. [94] proposed another method based on a heterogeneous network, using graph representation learning via substructures contrast. The dataset includes many relationships such as protein- disease- interaction, drug- protein- interaction, drug- disease- interaction, drug- drug- interaction, protein- protein- interaction, drug- structure- similarity, and protein- sequence- similarity. Thus, only the edges in the relationship set containing drug- drug– interaction, protein- protein– interaction will be converted into isomorphic matrices, and the drug– structure– similarity and protein– sequence– similarity will be converted into feature matrices, while the rest will be converted into heterogeneous matrices. The drug representation can be described as follow:

$$A = D^{-1/2}\hat{A}D^{-1/2} \tag{3.24}$$

where I is the identity matrix, and D is the degree matrix. Next is the GCN that encodes the nodes, and its representation is the following equation:

$$h = \text{Softmax}\left(AXW_1\right) \tag{3.25}$$

where X is the drug's feature matrix, W_1 denotes the trainable weight matrix, A denotes the drug network matrix produced by Laplace decomposition, and h denotes the drug view's node representation. After the node representation, graph representation is taking part using mutual information as a measure of quantifying their relationship. So, graph-level representation of the drugs is aggregating as follow:

$$s = \sigma\left(\frac{1}{n}\sum_{i=1}^{n} h_i\right) \tag{3.26}$$

where h_i is the i-th row vector of the node representation h, n is the number of row vectors, σ is the maximum pooling function, and s is the graph-level global representation. After that, a bi-linear function as the discriminator is applied which will be used in the cross-entropy loss between the node–level representation h and the graph-level global representation s. The novelty here is the use of substructure representation (S) that should be highly relevant to the graph-level representation (G). To achieve this, the Mentis algorithm is used

to extract k-subgraphs of the drug-drug relationship matrix, to construct subgraphs on the vertices of the graph. After randomly selecting k nodes for the k-th subgraph representation, a neural network maximizes the mutual information between S and G, and thus a loss function is calculated as follows:

$$L_k = \sum_{i=1}^{N} \log D\left(s, g_i\right) + \sum_{j=1}^{M} \log\left(1 - \left(D\left(s, g_j\right)\right)\right) \tag{3.27}$$

where g_i is the substructure representation of the k-th subgraph in a positive example pair, and g_j is the substructure representation of the k-th subgraph in a negative example pair. Finally, it reconstructs the final drug-protein matrix, but before that, a gradient update to optimize the loss function is required as follow:

$$\min \left[G - UW_3\left(VW_4^T\right)\right]^2 + L_r + L_k \tag{3.28}$$

$$M = UW_3\left(VW_4^T\right) \tag{3.29}$$

where G is the original matrix, U is the final learned drug-embedding representation, V is the final learned protein-embedding representation, W3 and W4 are the learnable weight matrix, and M is the final predicted drug-protein matrix.

NeoDTI [95] is the proposal of Wan et al., and aims to automatically learn a network topology-preserving node-level embedding from a graph that can be used to greatly facilitate the prediction of drug-target interactions. The three main steps are i) neighborhood information aggregation, ii) updating the node embedding, and iii) topology-preserving learning of the node embedding. For the first step, the neighborhood information aggregation for node v requires node embedding represented in a d-dimensional vector and an edge weight matrix, and it is described by the following equation:

$$a_v = \sum_{r \in R} \sum_{u \in N_r(v),} \frac{s(e)}{M_{v,r}} \sigma\left(W_r f^0(u) + b_r\right) \tag{3.30}$$

where $N_r(v) = \{u, u \in V, u \neq v, (u, v, r) \in E\}$ denotes the set of adjacent nodes connected to $v \in V$ through edges of type $r \in \mathbb{R}, \sigma(.)$ stands for a nonlinear activation function over a single-layer neural network parameterized by weights $W_r \in \mathbb{R}^{d \times d}$, a bias term $b_r \in \mathbb{R}_d$, and $M_{v,r} = P_u \in N_r(v), e = (u, v, r)s(e)$ stands for a normalization term, and the initial node embedding $f^0(u), \quad u \in N_r(v)$ is obtained through a random mapping. The second step's equation states that the new embedding of node $f_1(v)$ can be obtained using a single-layer neural network that is parameterized by weights $W_1 \in \mathbb{R}^{d \times 2d}$, a bias

term $b^1 \in \mathbb{R}^d$ and a ReLU maximization function $\sigma(.)$ to non-linearly transform the concatenation of the original embedding $f_0(v)$ and the neighborhood aggregation information a_v, and then normalized by its l_2 norm. The final step reconstructs the final output and should be similar as much as possible with the original edge weight. The following equation describes this step:

$$\min_{\substack{\{f^0(u),W^1,\sum_{r\in R}\sum_{u,v\in V_b} \\ b^1,W_r,b_r, \\ G_r,H_r,\mid u\in V,r\in R\}}} \left[s(e) - f^1(u)^\top G_r H_r^\top f^1(v)\right]^2 \tag{3.31}$$

where $G_r, H_r \in \mathbb{R}^{d\times k}$ are edge-type specific projection matrices. Finally, after calculation of the predicted interaction confidence score between drug node u and protein node v, the reconstructed drug-target interaction matrix can be described as:

$$W_{DTI_reconstruct} = F_{drug} G_r H_r^\top F_{\text{target}}^\top \tag{3.32}$$

DTiGEMS+ [96] is the proposal of Thafar et al. from the university of Kaust, Saudi Arabia, and exploits graph embedding, graph mining, and similarity-based techniques. The main objective of this method is to generate two different graphs to obtain features and calculate a binary classification for the DTI. Both graphs are divided into three steps. For the first graph (G1) the first step is to apply the forward similarity selection (FSS) procedure to all DD and TT similarities. FFS is a similarity selection technique that is a heuristic process (greed search) to obtain the best similarity combination of drug similarity matrices and target similarity matrices. The second step is to integrate these selected similarities using the SNF algorithm, which adds a single similarity to both fixed-length similarities and fuses them using SNF, and this procedure is repeated per round till it converges. Thus, the third step is to use the DD fused similarity, TT fused similarity, and the DTI training part to construct the graph.

The initial step in generating the feature representations for each node in the second graph is to apply node2vec to the basic heterogeneous network of drugs and targets interactions. They retained just the top-k related drugs (or targets) for each and eliminated all other edges. After eliminating all weak edges, the KNN similarity graphs for drugs and targets are reinforced with the training portion of DTIs and put into the node2vec model. The second step is to calculate cosine similarity for each drug-drug pair and target–target pair to construct two new matrices. After calculating cosine similarity, new edges could appear between pairs of drugs (or targets) based on the structural and topological similarities that don't have high similarity in the basic graph. The third step is to use the cosine similarities (drugs and targets), and

the DTI training part to construct the heterogeneous graph. The two weighted graphs were important to be created for the feature extraction for the drug-target path-scores. The path score is determined for each simple path that begins with the source node (i.e. drug) and ends at the target node (i.e. target protein) for each drug–target pair, and it is calculated as follow:

$$\text{score}\left(d_i, t_j\right) = \sum_{p=1}^{n} \prod \left(P_{\text{weights}}\right) \tag{3.33}$$

where P is the set of paths that connect $drug_i$ to $target_j$. They computed two characteristics for each meta-path (paths that have the same path structure) structure by summing all meta-path scores for that path structure and calculating the max of all meta-path scores beneath that path structure. Therefore, 12 path scores were extracted for each graph, from six path structures. Then a feature selection is applied by eliminating weak features. The number of unknown DTIs being much larger than the number of known DTIs, and thus, some techniques were applied, such as Random oversampling and SMOTE to face the imbalance problem. Feature vectors are generated with their labels for all drug-target pairs that are fed to the supervised machine learning model to predict the DTIs using multilayer perceptron, random forest and, adaptive boosting. Then the output of the classifier is the class label, which is either a positive or negative label.

Chapter 4

Methodology

In this chapter we describe how we chose the data, how the high class imbalance problem decreased and how the features from drug SMILES and protein sequences were extracted. Then, using pre-built methods and tools we transformed those features into tensors-embeddings. Finally, the two models are presented, which perform two different tasks. The first one is based on GEN convolution layer and Convolution 1D pertaining to the features of the drugs and targets, and the output being the predicted binding affinity (interaction). The second method that is being proposed is based on Auto-encoder, fulfilling the task of linking prediction on heterogeneous graph between drug nodes and protein nodes. The models are available publicly on GitHub[1].

4.1 Data Preparation

A DTI problem consists of drugs and targets in the form of a matrix. Moreover, the cells between each row and column represent the binary interaction. The dataset is extracted from the DTINet project, published on GitHub[2]. In this dataset, 16 different text files provide different information for the DTI problem. The dataset contains 708 drugs and 1512 proteins. For these drugs and proteins, it provides the interactions between them in binary form. Also, this dataset contains similarity matrices (real values) and interaction (binary values) matrices for drug-drug and protein-protein relations (Drug-Drug Interaction, Protein-Protein Interaction, Drug-Drug Similarity and Protein-Protein Similarity). Moreover, it contains some other data files like side-effects, diseases, interactions with drugs or proteins, but these are out of the scope of this thesis. It is important to note that the drugs, proteins, diseases, and side-effects are organized in the same order across all files, including name lists, ID mappings, and interaction/association matrices. The DTI dataset is quite sparse with high class imbalance because the positive values are only 1.923 when the negative values are 1.068.573.

[1] https://github.com/gmixoulis/MSc_Thesis_DTI

[2] https://github.com/luoyunan/DTINet/tree/master/data

4.1.1 Drug and Target Collection

The dataset we just described, besides the interaction, it does not provide any other information for the drugs or the proteins. Thus, using the Python urlib3 and Beautifulsoup[3] libraries, we retrieved the included drug ids and acquired their SMILES representations from the DrugBank[4] database. Similarly, for targets-proteins, we extracted their amino acid sequences in string format from the Uniprot[5] database following their ID. Amino acids are a collection of twenty different chemicals utilized to construct proteins. Proteins are composed of one or more polypeptide chains of amino acids. And those are the building blocks of proteins. The important thing about amino acids is that when they're linked together, they fold to make the final shape of the protein [97]. Moreover, UniProtKB canonical sequences are available in FASTA format, as additional manually curated isoform sequences described in UniProtKB/Swiss-Prot, which is the format that we used to extract the protein features. Each amino acid is represented with either three letters or, as in our case, with one letter of the English alphabet, and put together they form a sequence that includes the information of the protein's structure, as it is described by UniProt[6] and shown in Figure 4.1.

4.1.2 Drug and Protein Feature Extraction

The SMILES and the Protein Sequence do not provide enough information for the classification or regression task using graph embeddings. For that reason we found some already developed and tested open-source tools that provide us the embedding representation of SMILES and Protein Sequences. The first tool is BioVec from [98], which is an innovative way of representing biological sequences. This method was created for bioinformatics applications. It uses the skip-gram model of Word2Vec [99] for protein sequences, representing a single dense n-dimensional vector. Skip-gram is one of self-supervised learning techniques used to find the most related words for a given word. This strategy outperformed previous classification methods by 94 percent when tested on Swiss-Prot protein sequences from 7,027 protein families [98]. This model can estimate protein structure using only sequence data. This so-called embedding model can subsequently be utilized to ascertain various information about the proteins of interest. This model can also be used as pre-training for deep learning applications in bioinformatics.

[3] https://www.crummy.com/software/BeautifulSoup/
[4] https://go.drugbank.com/drugs
[5] https://www.uniprot.org/uniprot/
[6] www.uniprot.org/docs/userman.htm

One-letter code	Three-letter code	Amino-acid name
A	Ala	Alanine
R	Arg	Arginine
N	Asn	Asparagine
D	Asp	Aspartic acid
C	Cys	Cysteine
Q	Gln	Glutamine
E	Glu	Glutamic acid
G	Gly	Glycine
H	His	Histidine
I	Ile	Isoleucine
L	Leu	Leucine
K	Lys	Lysine
M	Met	Methionine
F	Phe	Phenylalanine
P	Pro	Proline
S	Ser	Serine
T	Thr	Threonine
W	Trp	Tryptophan
Y	Tyr	Tyrosine
V	Val	Valine
O	Pyl	Pyrrolysine
U	Sec	Selenocysteine
B	Asx	Aspartic acid or Asparagine
Z	Glx	Glutamic acid or Glutamine
X	Xaa	Any amino acid

FIGURE 4.1: Amino Acids

The second tool we used is OGB (Open Graph Benchmark), a collection of benchmark datasets, data loaders, and evaluators for graph machine learning, as shown in Figure 4.2. It is developed by Stanford University, and it is compatible with the PyTorch Geometric (PyG) tool. It provides a method called smile2graph that gets as input a SMILES and returns a graph (Data from PyG) object. This method uses the RDKit tool, which is a collection of chemoinformatics and machine-learning software written in C++ and Python. OGB was developed to make easier the dataset finding, the preprocessing and evaluation, which saves time from the Data Scientists and Machine Learning Engineers to focus solely on the ML model, as depicted in Figure 4.2. This tool uses RDKit functions to extract features based on information of the molecule's atoms (i.e., if it is aromatic, number of H,

number of radical electrons, etc.) and information of the molecule's bonds between the atoms (i.e., type of the bond, if it is conjugated, etc.). The features extracted from a SMILES are: i) features for the atoms, ii) features for the bonds, and iii) the number of bond pairs.



FIGURE 4.2: OGB Purpose

4.2 Machine Learning

Having extracted features for drugs and proteins, we followed two different machine learning approaches to predict the interaction between a drug and a target. Moreover, the training and prediction functions, which calculates the loss, are different for both approaches.

4.2.1 GENGRAM-DTA

The GENGRAM-DTA approach combines the GEN convolution, which is used for the drug low-dimensional representation and the N-GRAM, which is used to extract protein feature vectors.

The main idea of the GENeralized Aggregation Networks (GENConv) [100] is to keep all the message features to be positive so that generalized mean-max aggregation functions can be applied. The message construction function can be defined as follow:

$$\begin{aligned} \mathbf{m}_{vu}^{(l)} &= \rho^{(l)}\left(\mathbf{h}_v^{(l)}, \mathbf{h}_u^{(l)}, \mathbf{h}_{e_{vu}}^{(l)}\right) \\ &= \text{ReLU}\left(\mathbf{h}_u^{(l)} + \mathbb{1}\left(\mathbf{h}_{e_{vu}}^{(l)}\right) \cdot \mathbf{h}_{e_{vu}}^{(l)}\right) + \epsilon, u \in \mathcal{N}(v) \end{aligned} \tag{4.1}$$

where the ReLU function is a rectified linear unit [101] that outputs values to be greater or equal to zero and $\mathbb{1}(\cdot)$ is an indicator function being 1 when edge features exist otherwise 0, and ϵ is a small positive constant chosen as 10^{-7}. Furthermore, the $\rho^{(l)}$ is a learnable or differentiable function for message construction at the l-th layer. Thus, Message construction function $\rho^{(l)}$ is applied to vertex features $\mathbf{h}_v^{(l)}$ of v, its neighbor's features $\mathbf{h}_u^{(l)}$, and the corresponding edge features $\mathbf{h}_{e_{vu}}$ to construct an individual message $\mathbf{h}_{e_{vu}}$, for each neighbor $u \in \mathcal{N}(v)$. Moreover, GENConv follows a better pre-activation variant of residual connections for GCNs and uses a message normalization layer, which can significantly boost the performance of networks with under-performing aggregation functions by the normalization of

the features of the aggregated message combining them with other features during the vertex update phase.

For the model, first, we transformed the dataframe of the DTI matrix into a 3 column dataframe, the drugs, the targets, and the interaction. Then, a function iterates that dataframe and converts it into a PyG graph with batches (Data) of the drugs and targets. Then, the training starts in batches with Adam optimization [102] and a learning rate of 0,001. The model architecture is shown in Figure 4.3.

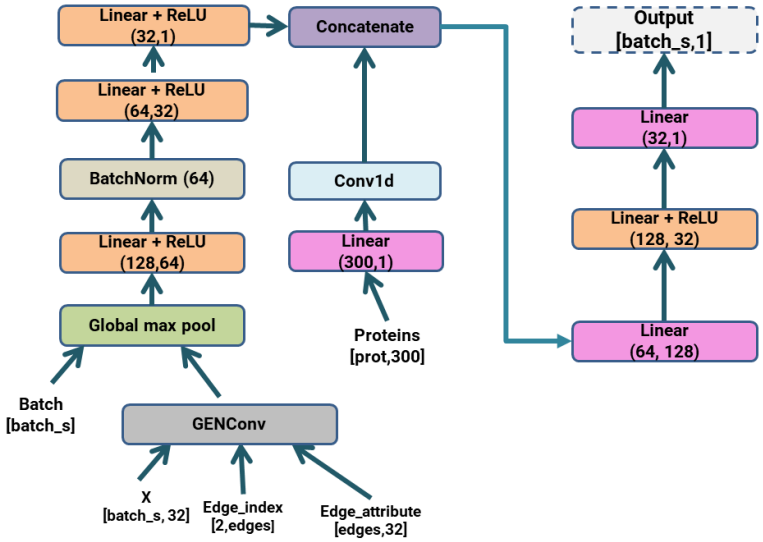


FIGURE 4.3: GENGRAM-DTA model Architecture

The model manages the drugs and targets differently. The drug features (atom features, edge features and edges between atoms) are inserted into a GEN Convolution, and returns the drug embedding. After that, the model normalizes the drug embedding using a BatchNorm1d, and the result of this are inserted to a global max pooling layer with the batch that created from DataLoader. The global max pooling is usually used to prevent overfiting on batches and returns batch-wise graph-level-outputs by taking the channel-wise maximum across the node dimension. The final drug embedding is computed using linear layers with the final shape to be [batch_size,1]. The batch size depends on the number of drug-target pairs. We had to compute 18,403 pairs for the training set thus, the batch size was set to 1,673. The batch size for the test set was set as 550 because the drug-target pairs were 2,750.

The protein features are already vectorized, thus the model uses 1D representations and layers of 1D convolutions to capture predictive patterns within the data. Finally, the two representation vectors are concatenated and passed through several fully connected layers to estimate the output drug–target binding affinity value.

We developed two different functions for the train and test set. The first is the training function which iterates over the batches from the DataLoader object and sends them one by one to the model. The model returns the affinity score of the drug-target pairs and save it into a tensor. To calculate the metrics later we also save the original interaction (affinity) in a tensor. Since we developed a regression method, even though we work with binary interactions, the loss is calculated using mean squared error (MSE) loss creating a criterion that measures the mean squared error (squared L2 norm) between each element in the input X and target Y. Then a function is called which computes $dloss/dx$ for every parameter x and requires gradients. These are accumulated into x.grad for every parameter x, in pseudo-code it is described as: $x.grad+ = dloss/dx$. Then, another function is called which updates the value of x using the gradient $x.grad$. For each epoch it is important to call the function that clears $x.grad$ for every parameter x in the optimizer. This is important to be called in the beginning of each epoch, otherwise it will accumulate the gradients from multiple passes. The prediction function is similar to the training one, but without optimization and processes the validation and test sets instead of the train set, which was used in the training function. The prediction function computes the metrics.

It is better to see the most abstract overview of this model which is depicted in Figure 4.4. The protein target is in the form of protein sequences, which are transformed using the BioVec tool. The output is a vector of 300 dimensions, which then is inserted into a Convolution layer that outputs the final protein tensor. Similarly, the drug with its chemical structure are transformed into a molecular-based graph of 32 dimension vectors (atoms, bonds, bond attributes) using the RDKit and the OGB tool, and using the GEN convolution to generate the final drug tensor. The above two vectors are concatenated and produce the binding affinity.

4.2.2 LP-DTI

To transform the drugs from a graph form to an embedding form, we used the DeeperGCN method, which is based on training very deep GCNs utilizing GEN Convolution. So, as shown in Figure 4.5 the input of the model is the drug features (atom features, bond features, bonds) and the output shape is [1,32].

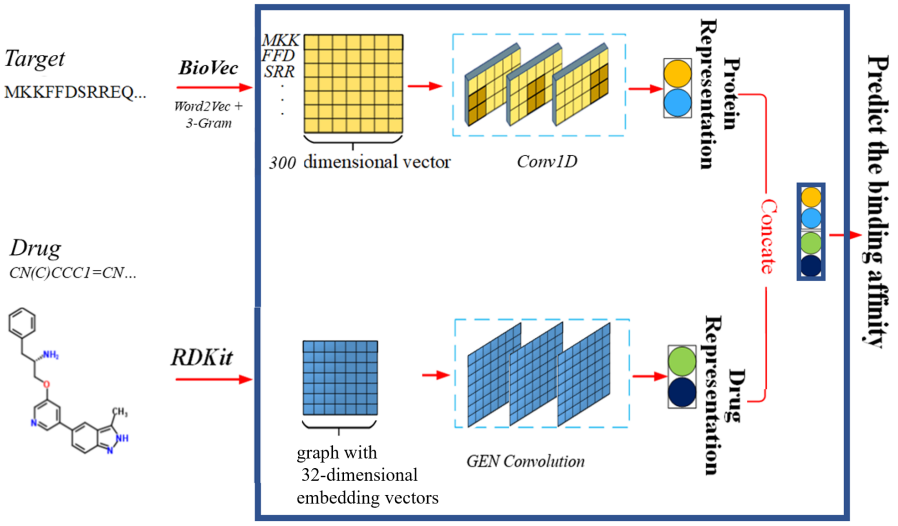


FIGURE 4.4: Abstract overview of the GENGRAM-DTA

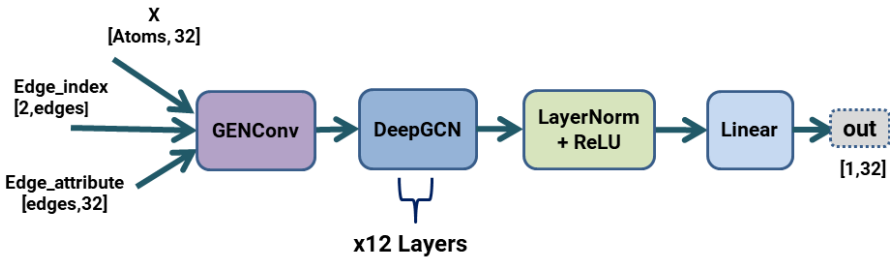


FIGURE 4.5: DeepGCN Architecture

The dataset we described in the first section was originated for auto-encoder purposes. Thus, we wanted to test it also in a similar model that uses encoders and decoders. This approach uses the dataset that includes only Drug Target Interaction matrix, as we described before. The drug and protein features are the same as the first approach. After converting the matrix above into dataframe, we transform it into lists of positive values (we kept only the values equal to 1). Thus, the heterogeneous graph was created using the PyG library, and the edges are shaped (2, number of edges) and are the following:

- Drug-Target-Interaction edges. These edges represent the positive only edges (the interactions), and the number of edges is 1,923.

The next thing in the graph making is to transform the graph into undirected form by using a PyG function. Then, using a random link split function from PyG that splits the edges, we get the train (0.8 of the whole data), validation (0.05 of the whole data), and test (0.15 of the whole data) data with a negative sampling of 1:10 ratio. Negative

sampling connects randomly with negative edge some nodes in the graph that do not have an edge between them.

After creating the graph, model training takes place. The model is a Graph Auto Encoder (GAE[7]) that includes an edge encoding and decoding system. The encoder takes the graph as input with information on the node features handled by the Graph convolution networks. Then, the encoder produces the embedding of the nodes in the latent space ($Z = \bar{X}$). Here, the encoder consists of two SAGE Convolution layers by adding neighbor information and normalizing the output features using $\frac{\mathbf{x}'_i}{\|\mathbf{x}'_i\|_2}$. The initialization for the convolution layers is lazy. Since the number of input features and thus the size of tensors varies between different types, PyG can make use of lazy initialization to initialize parameters in heterogeneous GNNs. This allows us to avoid calculating and keeping track of all tensor sizes of the computation graph. Then the decoder aims to reconstruct the input matrices, which is the concatenation of the drugs and proteins features, and then using a linear function, it outputs the final representation with shape (1,32). The abstract representation of the encoder and the decoder is depicted in Figure 4.6.

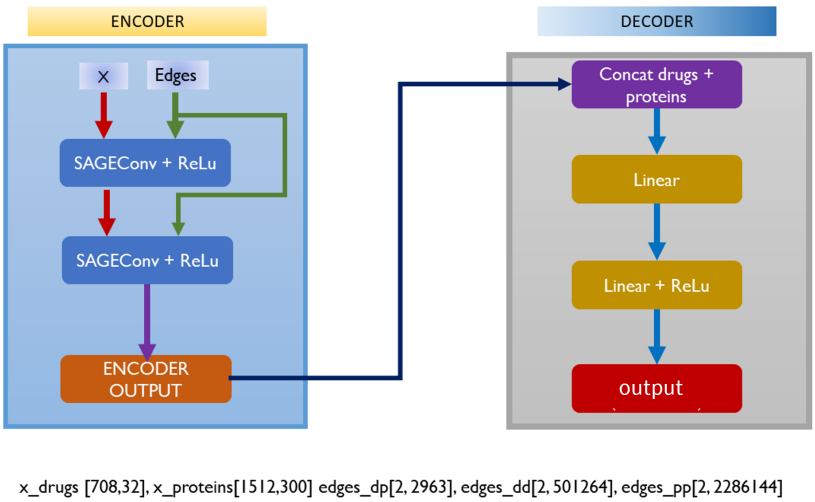


FIGURE 4.6: Abstract overview of the EncoderDecoder

Thus, the input for the model are three different dictionaries that were auto-created by the PyG, the dataset features dictionary (proteins and drugs), the data edge index, which are the edges between the nodes, and the labels of the data edges which are binary. The output is an array of features that are used for metrics score and show how well the model performs.

[7] https://github.com/pyg-team/pytorch_geometric/blob/master/examples/hetero/hetero_link_pred.py

An essential factor for the prediction task is the loss function, which is the binary cross-entropy with logits. This loss combines a Sigmoid layer and the Binary cross entropy loss (BCELogitsLoss) in one single class. This version is more numerically stable than using a plain Sigmoid followed by a BCELogitsLoss as, by combining the operations into one layer, we take advantage of the log-sum-exp trick for numerical stability. This loss function is described by the following equation:

$$l_n = -w_n \left[y_n \cdot \log x_n + (1 - y_n) \cdot \log (1 - x_n) \right] \tag{4.2}$$

where w_n are the weights, y_n is the target value and x_n is the predicted value. It compares the prediction, which is a number between 0 and 1, with the true target, that is either 0 or 1. It is a way to punish extremely wrong predictions more aggressively than ones that are close to the target. This stabilizes the training process. The optimization function is Adam, and the Learning rate is 0,0001. The abstract graphical representation of the model is shown in Figure 4.7, where the original DTI, which is also described in the interaction table, is different than the link predicted DTI, because the Link Prediction task finds new links between drugs and proteins.

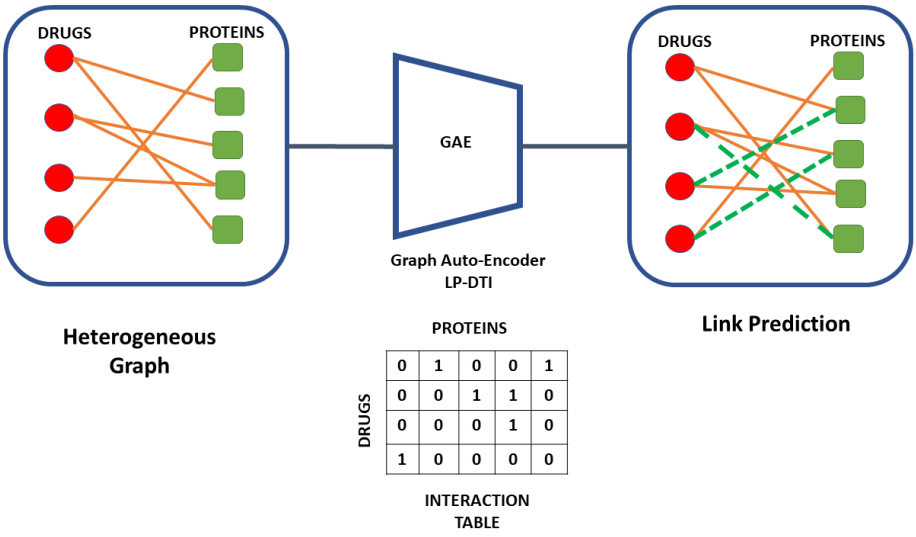


FIGURE 4.7: Abstract architecture of LP-DTI

Similar to the previous model, here we also developed two different functions for the training and prediction task. In a for loop we call both of the above functions for 100 epochs. The first is the training function, which sends the hetero-data to the model. The training function always starts with the initialization of the gradients. The hetero-data are the two types of nodes, the drugs and the proteins. PyG provides a feature that allows those hetero-data to be send to the model in a dictionary form, making it really easy to implement. Then it returns

the edge label which will be compared later to extract the metrics and the loss.

We picked this for two reasons: i) we have binary edge labels and we need to classify them, that is why we preferred the binary cross entropy, ii) binary cross entropy loss is suited for node classification and not for link prediction (the most suited for link prediction is the binary cross entropy with logits function [103]). The difference between those two loss functions is that the logit loss computes the sigmoid of the X factor instead of plain X.

The final two methods that are called on the training function are the backward gradients related function with retain_graph and the step() for the optimizer. The retain_graph will retain any necessary information to calculate a certain variable, so that we can do backward pass on it. The prediction (test) function follows the same procedure as the training, but without the optimization.

4.3 Results and Discussion

In the previous sections, we described the models we developed and the tools we used. In this section, we will present the evaluation metrics, learning curve, and the results of our proposed models.

4.3.1 Metrics

Evaluating the trained model's performance is a critical component of every machine learning effort. Different evaluation techniques are available for clustering, classification, and regression problems. This section discusses some of the most frequently used categorization evaluation metrics, such as rooted mean error score, square root mean error score, RocAUC, mean absolute error, spearman and pearson correlation. It should be emphasized, that as the DTI prediction problem suffers greatly from class imbalance, it is beneficial to choose class imbalance aware measures. As such, the most suitable of them is AUC.

- Area Under the ROC Curve (AUC) Receiver Operating Characteristic (ROC) Curve is a diagnostic plot for summarizing the behavior of a model, by calculating the true positive rate (or recall or sensitivity) and the false positive rate for a set of predictions by the model under different thresholds. The false positive rate is calculated as:

$$FPR = FP(FP + TN) \tag{4.3}$$

  The area beneath the receiver operating characteristic curve (AUR) is a two-dimensional area beneath the complete ROC curve from (0,0) to (1,1), as seen in Figure 4.8. AUC is a performance metric that aggregates performance across all categorization criteria.

AUC can be thought of as the likelihood that a random positive example is ranked higher than a random negative example by the model. AUC has a value between 0 and 1 and is expressed as a percentage. AUC of 0.0 indicates that the model is completely inaccurate, whereas AUC of 1.0 indicates that the model is completely accurate. Thus, in the Figure, 4.8 D is a perfect classifier, A and B, despite their errors, produce better predictions than chance, classifier C produces random predictions, and classifier E is worse than no-skill classifier. AUC is a good assessment metric since it is scale-invariant, as it assesses the ranking of predictions rather than their absolute values.

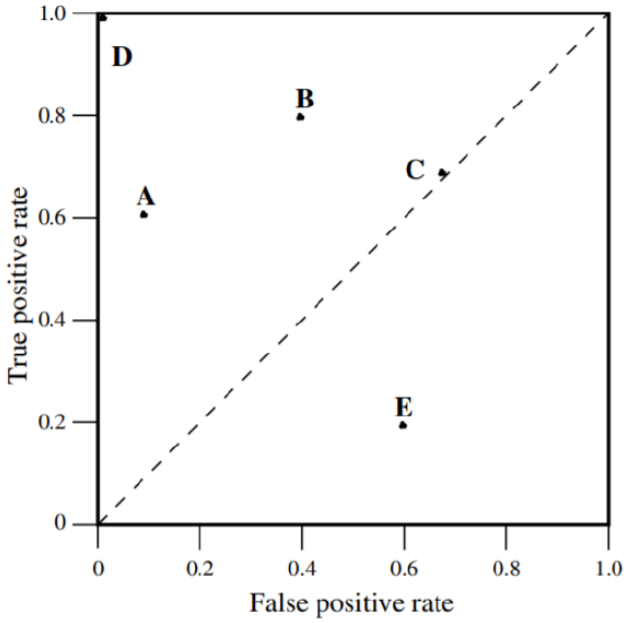


FIGURE 4.8: Area Under the Curve example [104]

Additionally, it is classification-threshold invariant, as it quantifies the model's predictive quality regardless of the classification threshold used.

- The Spearman rank- order correlation coefficient is a non- parametric measure of the monotonicity of the relationship between two variables. Unlike the Pearson correlation, the Spearman correlation does not assume that both variables are normally distributed. Like other correlation coefficients, this one varies between -1 and +1 with 0 implying no correlation. Correlations of -1 or +1 imply an exact monotonic relationship. Positive correlations imply that as x increases, so does y. Negative correlations

	Score Range
F1	$[0, 1]$ maximazing to 1
ROC-AUC	$[0, 1]$ maximazing to 1
SPEARMAN	$[-1, 1]$
PEARSON	$[-1, 1]$
RMSE/MSE	$[+\infty, 0]$ maximazing to 0

TABLE 4.1: Metrics Overview

imply that as x increases, y decreases. The p-value roughly indicates the probability of an uncorrelated system producing variables that have a Spearman correlation at least as extreme as the one computed from these datasets.

- The Pearson correlation coefficient measures the linear relationship between two datasets. The calculation of the p-value relies on the assumption that each dataset is normally distributed. Like other correlation coefficients, this one varies between -1 and +1 with 0 implying no correlation. Correlations of -1 or +1 imply an exact linear relationship.

- MSE for Mean Square Error, is a risk function that helps us determine the average squared difference between the predicted and the actual value of a feature or variable.

- RMSE is an acronym for Root Mean Square Error, which is the square root of value obtained from Mean Square Error function. Root Mean Square Error is the square root of the average of the squared differences between the estimated and the actual value of the variable/feature. Using RMSE, we can easily plot a difference between the estimated and actual values of a parameter of the model.

- The F measure or F score is a way of combining the precision and recall of the model, and it is defined as the harmonic mean of these two metrics. F1 is usually more useful than accuracy, especially when the data have an uneven class distribution.

- Mean absolute error (MAE) is a measure of errors between paired observations expressing the same phenomenon. In other words, the MAE measures the average magnitude of the errors in a set of forecasts, without considering their direction. It measures accuracy for continuous variables.

The Table 4.1 depicts the overview of score range for each metric. The left value inside the brackets is the lowest score and the right value is the highest score a metric can achieve.

4.3.2 GENGRAM-DTA

For the regression problem the metrics we usually get are the RMSE, MSE, MAE and sometimes the ROC-AUC. In addition, we also calculated the Spearman and Pearson correlation. The following table shows the best measures that our model achieved. The Figure 4.9 depicts the Learning curves which are the loss of each epoch for the training and validation set. Even though for the first 20 epochs we see that the model overfits as shown in the Figure 4.9, the model in the end reaches a good fit because in the 70th epoch and after, the valid loss has a small gap with the training loss and they reach a stability point.

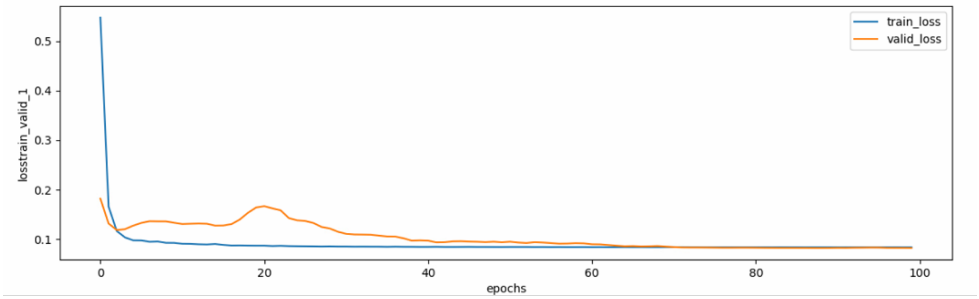


FIGURE 4.9: Train - Validation set learning curves

4.3.3 LP-DTI

For the LP-DTI which is a classification task, we used the same metric values as the previous task. The difference here is that the classification task value more the ROC-AUC and F1 score than RMSE and MSE score. The metrics we calculated are shown in the table 4.3 with the comparison results.

In general, plot of learning curves shows a good fit if:

- The plot of training loss decreases to a point of stability.
- The plot of validation loss decreases to a point of stability and has a small gap with the training loss.

Both of the above requirements are met for the LP-DTI's learning curves which are depicted in the following igure: 4.10. In addition, the learning curves has the same slope due to the nature of SAGE convolutions.

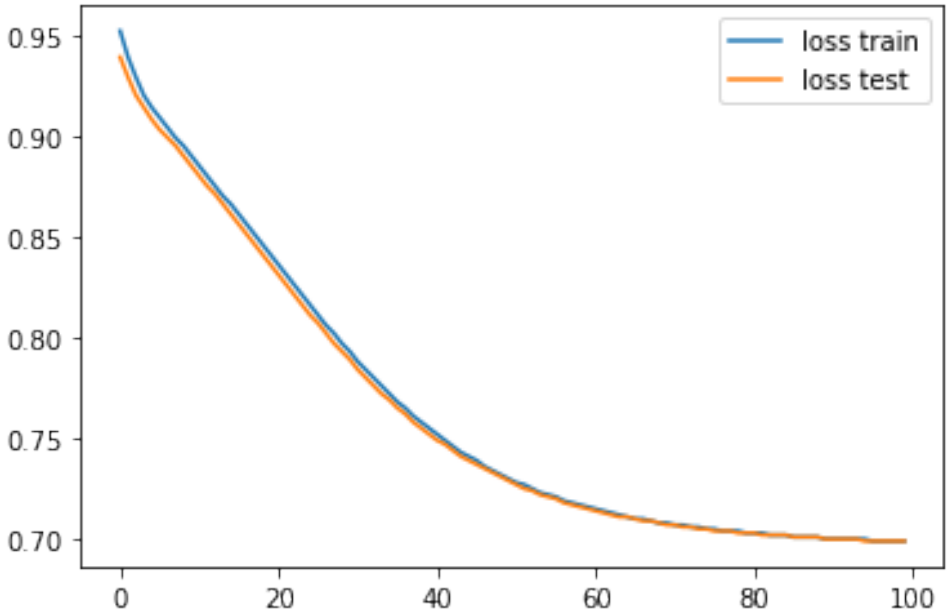


FIGURE 4.10: LP-DTI learning curves.

Finally, we developed a classification task with a DummyClassifier, using the sklearn tool. We called the model once without gradients to obtain the graph embedding in order to use them as the prediction variation (X) for the Dummyclassifier, while the target values (Y) were the original interactions. We used the most frequent value as an argument for the classifier. The predict method always returns the most frequent class label in the observed y argument passed to fit. The recorded score was 0.91.

4.3.4 Comparison with Related work

After obtaining the metrics for both approaches, we need to compare them with baselines to see how well they performed. The baselines we choose are already implemented ideas, published in scientific papers or conferences. For the binding affinity approach, we chose to compare with GraphDTA [105]. For the DTI approach, we choose EmbeddingDTI [106]. Firstly we will present the comparison results of the affinity task. In table 4.2, we describe the results of the four different models implemented by GraphDTA and the results of the model we proposed. The table shows that our approach achieves better scores in all metrics. GERNGRAM-DTA achieved RMSE of 0.21 while the best of the rest of the models achieved 0.27. Moreover, ROC-AUC is another important metric and we observe that GENGRAM-DTA achieved an outstanding score of 0.88 while the best of the rest of the models achieved 0.6. Also, in MAE, another important regression metric, our method achieves 0.11, while the rest of the models are

	GATNet	GAT_GCN	GCNet	GINConvNet	GENGRAM-DTA
RMSE	0.273	0.27	0.279	0.285	0.21
MSE	0.074	0.072	0.077	0.081	0.04
Roc-AUC	0.54	0.6	0.31	0.55	0.88
MAE	0.14	0.14	0.13	0.14	0.11
Pearson	0.049	0.1	-0.15	0.04	0.005
Spearman	0.042	0.11	-0.17	0.05	0.006

TABLE 4.2: GENGRAM-DTA comparison table

close to 0.14. The Spearman and Pearson correlation is not a metric to compare but provides valuable information for the correlation of the data. Thus, GAT_GCN proved to have the highest correlation while GENGRAM-DTA has the lowest correlation. There are three reasons why our approach performed better, i) we used more information to represent molecular graph (edge attributes), ii) GEN Convolution outperforming GAT and GIN convolutions, and iii) the combination of 3-Gram and Word2Vec to represent protein vectors were more successful than other approaches, such as using the word embedding of each protein sequence.

The LP- DTI is compared with the DeepEmbedding- DTI, a novel DTI implementation which is proven to be better than our implementation. The Link Prediction task is not popular in DTI task, thus we did not find any relevant link prediction work for DTI. The DeepEmbedding-DTI has better scores in any metric but f1-score, for which our model scored 0.9 whereas DeepEmbedding- DTI scored 0.64.

Overall, for binding affinity mode, the GENGRAM-DTA proved to have better evaluation metrics, while the second DTI model, LP-DTI, showed more limitations.

	ROC-AUC	RMSE	MSE	F1	Spearman
LP-DTI	0.783	0.29	0.08	0.91	0.22
DeepEmbedding-DTI	0.85	0.25	0.05	0.64	0.40

TABLE 4.3: LP-DTI comparison table

Chapter 5

Conclusions and Future Work

5.1 Conclusions

Discovering drug-target interactions is a critical stage in the drug design process. It is advantageous in the identification of novel medications, the repurposing of current pharmaceuticals to treat new diseases, or in the prediction of adverse effects during the early stages of drug development. While experimental validation of drug-target interactions is a critical step before compounds are selected for further development, it is also a time-consuming, costly, and tiresome process. As a result, it is critical to be in the position to correctly anticipate interactions prior to conducting wet-lab studies, in order to decrease drug development expenditures. While pre-existing computational methods attempted to tackle the challenge, machine learning approaches have dominated in recent years in terms of predicted performance.

Emerging in the field of machine learning DTI predictions, are the Graph Embedding using node features. The aforementioned techniques, in contrast to traditional approaches, seek to exploit graph representation learning of drugs and proteins, in order to uncover valuable information leading to the interactions prediction. This ever-increasing adoption of graph-based data is be owed to the good performance showed by relevant recent works, while being a field that has not been extensively researched yet. We have explored the field of graph representation learning using deep neural networks for the prediction of DTIs and binding affinity. The models we developed are using the link prediction method to predict interactions based on the graph topology and also we developed a method that uses graph-based methods to extract the node features and then using graph neural networks to predict the binding affinity, which is a task of regression.

Furthermore, we demonstrated that the GENConv provides better performance than other state-of-the-art methods for the binding affinity approach while the pre-processing in which we used word2vec with 3-gram to extract protein features and the graph embeddings on drug molecules using the RDKit tool to extract drug features, were proven better in performance. Moreover, the dataset we chose contain

binary interactions, which is not the optional option for the regression approach, thus the recorded metrics are not the best that this model could actually achieve. On the other hand, the link prediction task was not so successful even though it produced decent metrics. We believe that the model underfits and that's due to model incompetence to learn the graph structure of the data. A primary disadvantage of our method, is the large computational requirements and sparse imbalanced data. The possible DTIs in this dataset are over 1 million, which increased the time requirements exponentially. We sampled the negative (0) data to control the ratio between negative and positive data to be 10:1 meaning that, a large number of unknown interactions were excluded. Given the available computation resources, the model needed a lot of time to train, thus if we were to use a method that deals with the class imbalance by generating positives interaction samples (oversampling, SMOTE etc) it would be computationally impossible to train . In addition to this, each protein and drug is associated with a large number of features, while graph neural networks are generally considered computationally expensive. Another thing we concluded is that the current graph representation learning python tools are limited. The first tool we examined was the DGL tool[1]. This tool while it is quite fast in executing and processing information, it proved in the end incompatible with our data. It is very inflexible and has a limited number of graph convolution networks implemented. The graph supports node features only for the connected nodes, meaning that isolated nodes can not be implemented. The sparse dataset we chose had many isolated nodes, thus DGL could not be used. Pytorch geometric, on the other hand, is more flexible and has more complete documentation to explore, even though it is more time consuming during the training phase. Both of them show limitation regarding heterogeneous graphs, which are relevant in our DTI approach. The limitations concern the data features, for example each drug has different number of atoms and shape, but PyG on feature initialization requires each feature to have the same shape. To tackle this issue we used padding techniques, but it is still not a perfect solution.

5.2 Future work

This work touches only a few of the possible solutions for the DTI and binding affinity prediction using graph embedding and node features. Alternative data-driven solutions could enhance the prediction metrics. There are plenty of DTI datasets that are more updated and contain more data. For example while we were trying to download the drug SMILES for our dataset, some of them were deprecated, thus we

[1]https://www.dgl.ai/

update them with the new drug name and SMILE. A new dataset with more (positive) data would of course increase the model performance. Additionally, we can use other information related to drugs and proteins, such as side effects and disease interaction, which were not used in this thesis. Moreover, there are plenty of techniques we found in literature that can be apply in the DTI or binding affinity prediction. For example, in Chapter 2 we discussed about graph representation learning using GANs, this brand new and promising techniques would unveil a whole new branch of approaches for DTI prediction. Moreover, there are some graph convolution layers available that produce good performance and are used in many other DTI models, but were not applied in this thesis, like GIN convolutions or GAT convolutions.

Furthermore, we had many ideas to try which would be proven ground breaking. Everyone utilizes the Area Under the Curve (AUC) metric for binary classification, but nobody specifically targets it in their loss function. Rather than that, the majority of us use loss functions such as Binary Cross Entropy (BCE) or BCE with logit loss (BCEWithLogitsLoss) to forecast links. This works well in most cases, but we could possibly obtain better results if the loss function were more akin to the AUC. Due to the fundamental difference between BCE and AUC, BCE frequently misbehaves during the last layer of training, as we attempt to lead it towards the highest AUC score. Much of the AUC optimization occurs during hyper-parameter adjustment. Early Stopping becomes an uncomfortable necessity as the model may diverge sharply at any time from its high score. A variety of solutions do exist that optimize ROC-AUC that could predict better score metrics[2].

[2]https://github.com/iridiumblue/roc-star

Bibliography

[1] K. Sachdev and M. K. Gupta, "A comprehensive review of feature based methods for drug target interaction prediction", *Journal of Biomedical Informatics*, vol. 93, p. 103 159, 2019, ISSN: 1532-0464. DOI: https://doi.org/10.1016/j.jbi.2019.103159. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1532046419300772.

[2] L. Li and M. Cai, "Drug Target Prediction by Multi-View Low Rank Embedding", *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 16, no. 5, pp. 1712–1721, 2019. DOI: 10.1109/TCBB.2017.2706267.

[3] H. Shi, S. Liu, J. Chen, X. Li, Q. Ma, and B. Yu, "Predicting drug-target interactions using Lasso with random forest based on evolutionary information and chemical structure", *Genomics*, vol. 111, no. 6, pp. 1839–1852, 2019, ISSN: 0888-7543. DOI: https://doi.org/10.1016/j.ygeno.2018.12.007. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S088875431830466X.

[4] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online Learning of Social Representations", ser. KDD '14, New York, New York, USA: Association for Computing Machinery, 2014, pp. 701–710, ISBN: 9781450329569. DOI: 10.1145/2623330.2623732. [Online]. Available: https://doi.org/10.1145/2623330.2623732.

[5] H. Cai, V. W. Zheng, and K. C.-C. Chang, "A Comprehensive Survey of Graph Embedding: Problems, Techniques, and Applications", *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 9, pp. 1616–1637, 2018. DOI: 10.1109/TKDE.2018.2807452.

[6] Y. Luo, X. Zhao, J. Zhou, J. Yang, Y. Zhang, W. Kuang, J. Peng, L. Chen, and J. Zeng, "A network integration approach for drug-target interaction prediction and computational drug repositioning from heterogeneous information", en, *Nature Communications*, vol. 8, no. 1, p. 573, Jun. 2017, ISSN: 2041-1723. DOI: 10.1038/s41467-017-00680-8. [Online]. Available: https://www.nature.com/articles/s41467-017-00680-8.

[7] H. Öztürk, A. Özgür, and E. Ozkirimli, "DeepDTA: deep drug–target binding affinity prediction", *Bioinformatics*, vol. 34, no. 17, pp. i821–i829, Sep. 2018. DOI: 10.1093/bioinformatics/bty593. eprint: https://academic.oup.com/bioinformatics/article-pdf/34/17/i821/25702584/bty593.pdf. [Online]. Available: https://doi.org/10.1093/bioinformatics/bty593.

[8] F. Harary, *Graph Theory*. Reading, MA: Addison-Wesley, 1969.

[9] Hamilton, *Graph Representation Learning*, 3. Morgan and Claypool, vol. 14, pp. 1–159.

[10] M. McPherson, L. Smith-Lovin, and J. M. Cook, "Birds of a Feather: Homophily in Social Networks", *Annual Review of Sociology*, vol. 27, no. 1, pp. 415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415. eprint: https://doi.org/10.1146/annurev.soc.27.1.415. [Online]. Available: https://doi.org/10.1146/annurev.soc.27.1.415.

[11] C. Donnat, M. Zitnik, D. Hallac, and J. Leskovec, *Spectral Graph Wavelets for Structural Role Similarity in Networks*, 2018. [Online]. Available: https://openreview.net/forum?id=rJR2ylbRb.

[12] M. Nickel, K. Murphy, V. Tresp, and E. Gabrilovich, "A Review of Relational Machine Learning for Knowledge Graphs", *Proceedings of the IEEE*, vol. 104, no. 1, pp. 11–33, 2016, ISSN: 1558-2256. DOI: 10.1109/jproc.2015.2483592. [Online]. Available: http://dx.doi.org/10.1109/JPROC.2015.2483592.

[13] *Link prediction*, en, Oct. 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Link_prediction&oldid=1050609072 (visited on 10/25/2021).

[14] K. K. Teru, E. Denis, and W. L. Hamilton, *Inductive Relation Prediction by Subgraph Reasoning*, 2020. arXiv: 1911.06962 [cs.LG].

[15] X. Zheng, H. Ding, H. Mamitsuka, and S. Zhu, "Collaborative Matrix Factorization with Multiple Similarities for Predicting Drug-Target Interactions", ser. KDD '13, Chicago, Illinois, USA: Association for Computing Machinery, 2013, pp. 1025–1033, ISBN: 9781450321747. DOI: 10.1145/2487575.2487670. [Online]. Available: https://doi.org/10.1145/2487575.2487670.

[16] C. C. Aggarwal, *Data Mining: The Textbook*, en. Springer International Publishing, 2015, ISBN: 978-3-319-14141-1. DOI: 10.1007/978-3-319-14142-8. [Online]. Available: https://www.springer.com/gp/book/9783319141411 (visited on 10/19/2021).

[17] M. Agrawal, M. Zitnik, and J. Leskovec, "Large-scale analysis of disease pathways in the human interactome", 2017. arXiv: 1712.00843.

[18] S. Pandit, D. H. Chau, S. Wang, and C. Faloutsos, "Netprobe: A fast and scalable system for fraud detection in online auction networks", *16th International World Wide Web Conference, WWW2007*, Jan. 2007. DOI: 10.1145/1242572.1242600.

[19] L. A. Adamic and E. Adar, "Friends and neighbors on the Web", *Social Networks*, vol. 25, no. 3, pp. 211–230, 2003, ISSN: 0378-8733. DOI: https://doi.org/10.1016/S0378-8733(03)00009-1. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0378873303000091.

[20] E. A. Leicht, P. Holme, and M. E. J. Newman, "Vertex similarity in networks", *Phys. Rev. E*, vol. 73, p. 026 120, 2 2006. DOI: 10.1103/PhysRevE.73.026120. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevE.73.026120.

[21] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, en, p. 513.

[22] *Laplacian matrix*, en, Page Version ID: 1044393028, Sep. 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Laplacian_matrix&oldid=1044393028 (visited on 10/25/2021).

[23] P. D. Hoff, A. E. Raftery, and M. S. Handcock, "Latent Space Approaches to Social Network Analysis", *Journal of the American Statistical Association*, vol. 97, pp. 1090–1098, 2002.

[24] H. Robbins and S. Monro, "A Stochastic Approximation Method", *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951. DOI: 10.1214/aoms/1177729586. [Online]. Available: https://doi.org/10.1214/aoms/1177729586.

[25] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks", 2016. arXiv: 1607.00653 [cs.SI].

[26] C. Merkwirth and T. Lengauer, "Automatic Generation of Complementary Descriptors with Molecular Graph Networks", *Journal of Chemical Information and Modeling*, vol. 45, no. 5, pp. 1159–1168, Sep. 2005, Publisher: American Chemical Society, ISSN: 1549-9596. DOI: 10.1021/ci049613b. [Online]. Available: https://doi.org/10.1021/ci049613b (visited on 10/19/2021).

[27] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The Graph Neural Network Model", *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009. DOI: 10.1109/TNN.2008.2005605.

[28] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry", Proceedings of Machine Learning Research, vol. 70, D. Precup and Y. W. Teh, Eds., pp. 1263–1272, 2017. [Online]. Available: https://proceedings.mlr.press/v70/gilmer17a.html.

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition", pp. 770–778, 2016, ISSN: 1063-6919. DOI: 10.1109/CVPR.2016.90. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/CVPR.2016.90.

[30] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, "Deep Sets", 2018. arXiv: 1703.06114 [cs.LG].

[31] R. L. Murphy, B. Srinivasan, V. Rao, and B. Ribeiro, "Janossy Pooling: Learning Deep Permutation-Invariant Functions for Variable-Size Inputs", 2019. [Online]. Available: https://openreview.net/forum?id=BJluy2RcFm.

[32] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, _eprint: https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.173 ISSN: 0899-7667. DOI: 10.1162/neco.1997.9.8.1735. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735.

[33] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate", *CoRR*, vol. abs/1409.0473, 2015.

[34] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks", in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=rJXMpikCZ.

[35] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All you Need", *ArXiv*, vol. abs/1706.03762, 2017.

[36] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2019.

[37] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, "Gated Graph Sequence Neural Networks", *CoRR*, vol. abs/1511.05493, 2016.

[38] K. Cho, B. van Merrienboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", 2014.

[39] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, "Representation Learning on Graphs with Jumping Knowledge Networks", in *Proceedings of the 35th International Conference on Machine Learning*, J. Dy and A. Krause, Eds., ser. Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 5453–5462. [Online]. Available: https://proceedings.mlr.press/v80/xu18c.html.

[40] T. N. Kipf and M. Welling, *Semi-Supervised Classification with Graph Convolutional Networks*, 2017. arXiv: 1609.02907 [cs.LG].

[41] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs", *arXiv:1706.02216 [cs, stat]*, Sep. 2018, arXiv: 1706.02216. [Online]. Available: http://arxiv.org/abs/1706.02216.

[42] M. Schlichtkrull, T. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling Relational Data with Graph Convolutional Networks", English, in *The Semantic Web - 15th International Conference, ESWC 2018, Proceedings*, ser. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), 15th International Conference on Extended Semantic Web Conference, ESWC 2018 ; Conference date: 03-06-2018 Through 07-06-2018, Springer/Verlag, 2018, pp. 593–607, ISBN: 9783319934167. DOI: 10.1007/978-3-319-93417-4_38.

[43] K. Sinha, S. Sodhani, J. Dong, J. Pineau, and W. L. Hamilton, *CLUTRR: A Diagnostic Benchmark for Inductive Reasoning from Text*, 2019. arXiv: 1908.06177 [cs.LG].

[44] O. Vinyals, S. Bengio, and M. Kudlur, "Order Matters: Sequence to sequence for sets", *CoRR*, vol. abs/1511.06391, 2016.

[45] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[46] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep Graph Infomax", in *International Conference*

on Learning Representations, 2019. [Online]. Available: https://openreview.net/forum?id=rklz9iAcKQ.

[47] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, *InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization*, 2020. arXiv: 1908.01000 [cs.LG].

[48] D. Selsam, M. Lamm, B. Bünz, P. Liang, L. de Moura, and D. L. Dill, *Learning a SAT Solver from Single-Bit Supervision*, 2019. arXiv: 1802.03685 [cs.AI].

[49] P. Erdős and A. Rényi, "On the Evolution of Random Graphs", in *PUBLICATION OF THE MATHEMATICAL INSTITUTE OF THE HUNGARIAN ACADEMY OF SCIENCES*, 1960, pp. 17–61.

[50] M. Newman, *Networks: An Introduction*. USA: Oxford University Press, Inc., 2010, ISBN: 0199206651.

[51] R. Albert and A.-L. Barabási, "Statistical mechanics of complex networks", *Rev. Mod. Phys.*, vol. 74, pp. 47–97, 1 2002. DOI: 10.1103/RevModPhys.74.47. [Online]. Available: https://link.aps.org/doi/10.1103/RevModPhys.74.47.

[52] T. Kipf and M. Welling, "Variational Graph Auto-Encoders", *ArXiv*, vol. abs/1611.07308, 2016.

[53] M. Simonovsky and N. Komodakis, "GraphVAE: Towards Generation of Small Graphs Using Variational Autoencoders", in *ICANN*, 2018.

[54] W. Jin, R. Barzilay, and T. Jaakkola, "Junction Tree Variational Autoencoder for Molecular Graph Generation", in *Proceedings of the 35th International Conference on Machine Learning*, J. Dy and A. Krause, Eds., ser. Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 2323–2332. [Online]. Available: https://proceedings.mlr.press/v80/jin18a.html.

[55] A. Brock, J. Donahue, and K. Simonyan, "Large Scale GAN Training for High Fidelity Natural Image Synthesis", 2019. arXiv: 1809.11096 [cs.LG].

[56] N. D. Cao and T. Kipf, "MolGAN: An implicit generative model for small molecular graphs", 2018. arXiv: 1805.11973 [stat.ML].

[57] K. Sachdev and M. K. Gupta, "A comprehensive review of feature based methods for drug target interaction prediction", en, *Journal of Biomedical Informatics*, vol. 93, p. 103 159, May 2019, ISSN: 15320464. DOI: 10.1016/j.jbi.2019.103159. [Online].

Available: https://linkinghub.elsevier.com/retrieve/pii/S1532046419300772 (visited on 08/17/2021).

[58] *Definition of drug | Dictionary.com*, en. [Online]. Available: https://www.dictionary.com/browse/drug (visited on 08/18/2021).

[59] *Drug Target - an overview | ScienceDirect Topics*. [Online]. Available: https://www.sciencedirect.com/topics/chemistry/drug-target (visited on 08/18/2021).

[60] N. Calvey, "Enzyme inducers and inhibitors: addition, subtraction and synergism", *Anaesthesia & Intensive Care Medicine*, vol. 6, pp. 139–140, 2005, ISSN: 1472-0299. DOI: https://doi.org/10.1383/anes.6.4.139.63630. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1472029906700704.

[61] *Enzyme Inhibitor - an overview | ScienceDirect Topics*. [Online]. Available: https://www.sciencedirect.com/topics/chemistry/enzyme-inhibitor.

[62] T. Takenaka, "Classical vs reverse pharmacology in drug discovery", en, *BJU International*, vol. 88, no. s2, pp. 7–10, 2001, _eprint: https://bjui-journals.onlinelibrary.wiley.com/doi/pdf/10.1111/j.1464-410X.2001.00112.x, ISSN: 1464-410X. DOI: 10.1111/j.1464-410X.2001.00112.x. [Online]. Available: https://bjui-journals.onlinelibrary.wiley.com/doi/abs/10.1111/j.1464-410X.2001.00112.x (visited on 08/19/2021).

[63] H. Chen and Z. Zhang, "A Semi-Supervised Method for Drug-Target Interaction Prediction with Consistency in Networks", *PLOS ONE*, vol. 8, no. 5, pp. 1–7, May 2013, Publisher: Public Library of Science. DOI: 10.1371/journal.pone.0062975. [Online]. Available: https://doi.org/10.1371/journal.pone.0062975.

[64] S. Paul, D. Mytelka, C. Dunwiddie, C. Persinger, B. Munos, S. Lindborg, and A. Schacht, "How to Improve R&D Productivity: The Pharmaceutical Industry's Grand Challenge", *Nature reviews. Drug discovery*, vol. 9, pp. 203–14, Feb. 2010. DOI: 10.1038/nrd3078.

[65] K. M. Giacomini, R. M. Krauss, D. M. Roden, M. Eichelbaum, M. R. Hayden, and Y. Nakamura, "When good drugs go bad", en, *Nature*, vol. 446, no. 7139, pp. 975–977, Apr. 2007, Bandiera_abtest: a Cg_type: Nature Research Journals Number: 7139 Primary_atype: Comments & Opinion Publisher: Nature Publishing Group, ISSN: 1476-4687. DOI: 10.1038/446975a. [Online].

Available: https://www.nature.com/articles/446975a (visited on 08/22/2021).

[66] E. Pauwels, V. Stoven, and Y. Yamanishi, "Predicting drug side-effect profiles: a chemical fragment-based approach", *BMC Bioinformatics*, vol. 12, no. 1, p. 169, May 2011, ISSN: 1471-2105. DOI: 10.1186/1471-2105-12-169. [Online]. Available: https://doi.org/10.1186/1471-2105-12-169 (visited on 08/22/2021).

[67] Y. Wang, S. H. Bryant, T. Cheng, J. Wang, A. Gindulyte, B. A. Shoemaker, P. A. Thiessen, S. He, and J. Zhang, "PubChem BioAssay: 2017 update", eng, *Nucleic Acids Research*, vol. 45, no. D1, 2017, ISSN: 1362-4962. DOI: 10.1093/nar/gkw1118.

[68] A. Ezzat, M. Wu, X.-L. Li, and C.-K. Kwoh, "Computational prediction of drug–target interactions using chemogenomic approaches: an empirical survey", *Briefings in bioinformatics*, vol. 20, no. 4, pp. 1337–1357, 2019.

[69] M. A. Johnson and G. M. Maggiora, *Concepts and applications of molecular similarity*. Wiley, 1990.

[70] M. J. Keiser, B. L. Roth, B. N. Armbruster, P. Ernsberger, J. J. Irwin, and B. K. Shoichet, "Relating protein pharmacology by ligand chemistry", *Nature biotechnology*, vol. 25, no. 2, pp. 197–206, 2007, ISSN: 1087-0156. DOI: 10.1038/nbt1284. [Online]. Available: https://doi.org/10.1038/nbt1284.

[71] E. Byvatov, U. Fechner, J. Sadowski, and G. Schneider, "Comparison of Support Vector Machine and Artificial Neural Network Systems for Drug/Nondrug Classification", *Journal of Chemical Information and Computer Sciences*, vol. 43, no. 6, pp. 1882–1889, Nov. 2003, Publisher: American Chemical Society, ISSN: 0095-2338. DOI: 10.1021/ci0341161. [Online]. Available: https://doi.org/10.1021/ci0341161.

[72] L. Jacob and J.-P. Vert, "Protein-ligand interaction prediction: an improved chemogenomics approach", *Bioinformatics*, vol. 24, pp. 2149–2156, 2008.

[73] T. Cheng, Q. Li, Z. Zhou, Y. Wang, and S. Bryant, "Structure-Based Virtual Screening for Drug Discovery: a Problem-Centric Review", *The AAPS journal*, vol. 14, pp. 133–41, Mar. 2012. DOI: 10.1208/s12248-012-9322-0.

[74] H. Li, Z. Gao, L. Kang, H. Zhang, K. Yang, K. Yu, X. Luo, W. Zhu, K. Chen, J. Shen, X. Wang, and H. Jiang, "TarFisDock: A web

server for identifying drug targets with docking approach", *Nucleic acids research*, vol. 34, W219–24, Aug. 2006. DOI: 10.1093/nar/gkl114.

[75] G. P. M. V. A. A. C. B. M. J. S. M. B. J. Fernandez-Larrea and L. Arola, "Protein-ligand Docking: A Review of Recent Advances and Future Perspectives", *Current Pharmaceutical Analysis*, vol. 4, no. 1, pp. 1–19, 2008, ISSN: 1573-4129/1875-676X. DOI: 10.2174/157341208783497597. [Online]. Available: http://www.eurekaselect.com/node/66325/article.

[76] M. Yildirim, K.-I. Goh, M. Cusick, A.-L. Barabasi, and M. Vidal, "Drug-Target Network", *Nature biotechnology*, vol. 25, pp. 1119–26, Nov. 2007. DOI: 10.1038/nbt1338.

[77] S. J. Opella, "Structure determination of membrane proteins by nuclear magnetic resonance spectroscopy", *Annual Review of Analytical Chemistry*, vol. 6, pp. 305–328, 2013.

[78] Y. Yamanishi, M. Araki, A. Gutteridge, W. Honda, and M. Kanehisa, "Prediction of drug–target interaction networks from the integration of chemical and genomic spaces", *Bioinformatics*, vol. 24, no. 13, pp. i232–i240, Jul. 2008, ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btn162. eprint: https://academic.oup.com/bioinformatics/article-pdf/24/13/i232/629661/btn162.pdf. [Online]. Available: https://doi.org/10.1093/bioinformatics/btn162.

[79] W. Wang, S. Yang, and J. Li, "Drug target predictions based on heterogeneous graph inference", *Pacific Symposium on Biocomputing. Pacific Symposium on Biocomputing*, vol. 18, pp. 53–64, Feb. 2013.

[80] M. Liu and G. Yan, "Drug-target interaction prediction by random walk on the heterogeneous network", *Molecular bioSystems*, vol. 8, pp. 1970–8, Apr. 2012. DOI: 10.1039/c2mb00002d.

[81] S. Fakhraei, B. Huang, L. Raschid, and L. Getoor, "Network-Based Drug-Target Interaction Prediction with Probabilistic Soft Logic", *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 11, Sep. 2014. DOI: 10.1109/TCBB.2014.2325031.

[82] W. Ba Alawi, O. Soufan, M. Essack, P. Kalnis, and V. B. Bajic, *DASPfind: new efficient method to predict drug–target interactions*, 2016. DOI: 10.1186/s13321-016-0128-4. [Online]. Available: http://hdl.handle.net/10754/602276.

[83] Z. Yu, J. Lu, Y. Jin, and Y. Yang, "KenDTI: An Ensemble Model for Predicting Drug-Target Interaction by Integrating Multi-Source Information", *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 18, no. 4, pp. 1305–1314, 2021. DOI: 10.1109/TCBB.2021.3074401.

[84] Z. Liu, Q. Chen, W. Lan, H. Pan, X. Hao, and S. Pan, "GADTI: Graph Autoencoder Approach for DTI Prediction From Heterogeneous Network", *Frontiers in Genetics*, vol. 12, p. 490, 2021. DOI: 10.3389/fgene.2021.650821. [Online]. Available: https://www.frontiersin.org/article/10.3389/fgene.2021.650821.

[85] B. Yang, W.-t. Yih, X. He, J. Gao, and L. Deng, *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*, 2015. arXiv: 1412.6575 [cs.CL].

[86] J. Lim, S. Ryu, K. Park, Y. J. Choe, J. Ham, and W. Y. Kim, "Predicting Drug–Target Interaction Using a Novel Graph Neural Network with 3D Structure-Embedded Graph Representation", *Journal of Chemical Information and Modeling*, vol. 59, no. 9, pp. 3981–3988, 2019, PMID: 31443612. DOI: 10.1021/acs.jcim.9b00387. eprint: https://doi.org/10.1021/acs.jcim.9b00387. [Online]. Available: https://doi.org/10.1021/acs.jcim.9b00387.

[87] W. Chen, G. Chen, L. Zhao, and C. Y.-C. Chen, "Predicting Drug–Target Interactions with Deep-Embedding Learning of Graphs and Sequences", *The Journal of Physical Chemistry A*, vol. 125, no. 25, pp. 5633–5642, 2021, PMID: 34142824. DOI: 10.1021/acs.jpca.1c02419. eprint: https://doi.org/10.1021/acs.jpca.1c02419. [Online]. Available: https://doi.org/10.1021/acs.jpca.1c02419.

[88] S. Wang, P. Shan, Y. Zhao, and L. Zuo, "GanDTI: A multi-task neural network for drug-target interaction prediction", *Computational Biology and Chemistry*, vol. 92, p. 107 476, 2021, ISSN: 1476-9271. DOI: https://doi.org/10.1016/j.compbiolchem.2021.107476. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1476927121000438.

[89] H. Wang, G. Zhou, S. Liu, J.-Y. Jiang, and W. Wang, *Drug-Target Interaction Prediction with Graph Attention networks*, 2021. eprint: 2107.06099.

[90] B.-W. Zhao, Z.-H. You, L. Hu, Z.-H. Guo, L. Wang, Z.-H. Chen, and L. Wong, "A Novel Method to Predict Drug-Target Interactions Based on Large-Scale Graph Representation Learning",

Cancers, vol. 13, no. 9, 2021, ISSN: 2072-6694. DOI: 10.3390/cancers13092111. [Online]. Available: https://www.mdpi.com/2072-6694/13/9/2111.

[91] Z. Sajadi, M. A. Zare Chahooki, S. Gharaghani, and K. Abbasi, "AutoDTI++: deep unsupervised learning for DTI prediction by autoencoders", *BMC Bioinformatics*, vol. 22, Apr. 2021. DOI: 10.1186/s12859-021-04127-2.

[92] Y. Chu, X. Shan, T. Chen, M. Jiang, Y. Wang, Q. Wang, D. Salahub, Y. Xiong, and D.-Q. Wei, "DTI-MLCD: predicting drug-target interactions using multi-label learning with community detection method", *Briefings in bioinformatics*, vol. 22, Sep. 2020. DOI: 10.1093/bib/bbaa205.

[93] M. Xu, X. Zhang, and X. Lin, "Inferring Drug-Target Interactions Using Graph Isomorphic Network and Word Vector Matrix", in *2020 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, 2020, pp. 503–506. DOI: 10.1109/BIBM49941.2020.9313441.

[94] S. Cheng, L. Zhang, B. Jin, Q. Zhang, and X. Lu, "Drug Target Prediction Using Graph Representation Learning via Substructures Contrast", Mar. 2021. DOI: 10.20944/preprints202103.0337.v1.

[95] F. Wan, L. Hong, A. Xiao, T. Jiang, and J. Zeng, "NeoDTI: neural integration of neighbor information from a heterogeneous network for discovering new drug–target interactions", *Bioinformatics*, vol. 35, no. 1, pp. 104–111, Jul. 2018, ISSN: 1367-4803. DOI: 10.1093/bioinformatics/bty543. eprint: https://academic.oup.com/bioinformatics/article-pdf/35/1/104/27182907/bty543.pdf. [Online]. Available: https://doi.org/10.1093/bioinformatics/bty543.

[96] M. Thafar, R. Olayan, H. Ashoor, S. Albaradei, V. Bajic, X. Gao, T. Gojobori, and M. Essack, "DTiGEMS+: Drug-target interaction prediction using graph embedding, graph mining, and similarity-based techniques", *Journal of Cheminformatics*, vol. 12, Jun. 2020. DOI: 10.1186/s13321-020-00447-2.

[97] *Amino Acids*. [Online]. Available: https://www.genome.gov/genetics-glossary/Amino-Acids (visited on 02/01/2022).

[98] E. Asgari and M. R. K. Mofrad, "Continuous Distributed Representation of Biological Sequences for Deep Proteomics and Genomics", *PLOS ONE*, vol. 10, no. 11, F. H. Kobeissy, Ed., e0141287, 2015, ISSN: 1932-6203. DOI: 10.1371/journal.pone.0141287.

[Online]. Available: http://dx.doi.org/10.1371/journal.pone.0141287.

[99] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space", *arXiv:1301.3781 [cs]*, Sep. 2013, arXiv: 1301.3781. [Online]. Available: http://arxiv.org/abs/1301.3781 (visited on 03/14/2022).

[100] G. Li, C. Xiong, A. Thabet, and B. Ghanem, *DeeperGCN: All You Need to Train Deeper GCNs*, 2020. arXiv: 2006.07739 [cs.LG].

[101] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines", in *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ser. ICML'10, Haifa, Israel: Omnipress, 2010, pp. 807–814, ISBN: 9781605589077.

[102] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, 2017. arXiv: 1412.6980 [cs.LG].

[103] *Pytorch Geometric Tutorial*. [Online]. Available: https://antoniolonga.github.io/Pytorch_geometric_tutorials/posts/post12.html (visited on 03/02/2022).

[104] T. Fawcett, "An introduction to ROC analysis", *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, ROC Analysis in Pattern Recognition, ISSN: 0167-8655. DOI: https://doi.org/10.1016/j.patrec.2005.10.010. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S016786550500303X.

[105] T. Nguyen, H. Le, T. P. Quinn, T. Nguyen, T. D. Le, and S. Venkatesh, "GraphDTA: predicting drug–target binding affinity with graph neural networks", en, *Bioinformatics*, vol. 37, no. 8, M. Pier Luigi, Ed., pp. 1140–1147, 2021, ISSN: 1367-4803, 1460-2059. DOI: 10.1093/bioinformatics/btaa921. [Online]. Available: https://academic.oup.com/bioinformatics/article/37/8/1140/5942970.

[106] Y. Jin, J. Lu, R. Shi, and Y. Yang, "EmbedDTI: Enhancing the Molecular Representations via Sequence Embedding and Graph Convolutional Network for the Prediction of Drug-Target Interaction", *Biomolecules*, vol. 11, no. 12, 2021, ISSN: 2218-273X. DOI: 10.3390/biom11121783. [Online]. Available: https://www.mdpi.com/2218-273X/11/12/1783.